



Ministry of Higher Education
Modern Academy
Computer Science and Management
Technology in Maadi
Computer Science Department



Unheard Voices

Submitted to

Modern Academy

Computer Science and Management Technology in Maadi

Computer Science Department

Prepared by

Khaled Mohamed Nagy Mahmoud

Osama Gamal Hamed Ramadan

Amr Sabry Mohamed Elbery

Supervised by

Dr. Nora Niazy

Eng. Ahmed Fouad

Academic Year :2023/2024

Team Members:

Khaled Mohamed Nagy Mahmoud

Osama Gamal Hamed Ramadan

Amr Sabry Mohamed Elbery

Acknowledgement

Firstly, we would like to express our gratitude towards our university “Modern Academy” and its board members for their professionalism, supervision, and support that helped us grow on our educational journey.

“Prof. Dr. Mahmoud Gadallah” For his guidance and keenness on providing us with the best educational facilities along with the department staff that helped us so much throughout our academic years.

“Dr. Nora Niazy” For her valuable guidance and support in completion of this project. And for always pushing us to do better. We are truly thankful to have been given the opportunity to learn from her and be inspired by her encouragement since day one.

Finally, special thanks to “Eng. Ahmed Fouad” for his help and effort through the year and for always guiding us with patience and care. All the help and support that we have received during the project

implementation is very much appreciated and it is the reason we

were finally able to present our idea and make it real, hoping it helps many people out there.

Abstract

The "Unheard Voices" project is a pioneering initiative designed to tackle the profound communication obstacles confronting individuals who are deaf or mute, with a core focus on fostering their active participation in spontaneous dialogues and alleviating feelings of isolation. Through the strategic utilization of cutting-edge technologies, this project introduces an all-encompassing platform aimed at enhancing communication across diverse settings, including daily interactions, educational environments, and professional contexts. The primary objectives of the project are as follows:

Enhanced Communication: This project endeavors to offer a holistic solution that bridges the communication chasm between individuals with hearing or speech impairments and those who communicate verbally, thus enabling seamless interactions across various scenarios and environments.

Accurate and Real-Time Translation: Ensuring the precise and instantaneous translation of sign language gestures into either text or speech, thereby enabling users to comprehend and respond to communication stimuli without any perceptible delay.

Sign Language Education: Providing accessible tools and resources for both hearing and deaf individuals to acquire proficiency in sign language, thereby fostering mutual comprehension and facilitating smoother communication exchanges.

Educational Integration: Enabling students with hearing impairments to fully integrate into mainstream educational settings alongside their peers, thus obviating the necessity for segregated educational institutions.

Central to the platform's functionality are technological advancements such as sophisticated speech recognition algorithms, robust sign language interpretation models, and seamless text-to-speech synthesis capabilities. These pivotal features collectively empower users to engage in meaningful conversations, receive and process information in real-time, and enhance their sign language proficiency through interactive learning modules. By prioritizing the development of a user-friendly interface and incorporating comprehensive accessibility features, "Unheard Voices" endeavors to champion inclusivity and embolden individuals with hearing and speech impairments to communicate with greater efficacy and confidence. Through this transformative initiative, we aspire to create a more equitable and inclusive society where every voice is heard and valued.

Structure of the document:

This document will be split into Six main chapters.

Chapter 1: Introduction.

Chapter 2: Survey and Historical data.

Chapter 3: Software life cycle and Analysis.

Chapter 4: Tools.

Chapter 5: System Implementation

Chapter 6: Conclusion and future work.

Table of contents:

Chapter One (Introduction):

1.1 Introduction.....	12
1.2 Problem definition	14
1.3 Solution approach	16
1.4 Project Objective	17
1.5 Stakeholders	18
1.6 Scope.....	19
1.7 Constraints.....	20
1.8 Advantages.....	21

Chapter two (Survey):

2.1 Literature rereview.....	24
2.1.1 Ava application.....	25
2.1.2 Signly.....	27
2.1.3 SignSchool.....	28

Chapter three (Software life cycle):

3.1 Project Development Methodology (phases)	30
3.2 Planning	33
3.3 Analysis	34
3.3.1 Requirements Analysis	34
3.3.2 System Requirements	35

3.3.3 Software Requirements Specification	36
3.3.4 System Users	38
3.4.1 Use Case Diagram	39
3.4.2 Activity Diagram.....	40
3.4.3 Sequence Diagram.....	41
3.4.4 Class Diagram.....	42
3.4.5 Data flow diagram level 0.....	43
3.4.5.1 Data flow diagram level 1.....	44
3.4.5. Data flow diagram level 2.....	45
3.3.5 Tools	46
3.4 System Design	47
Chapter four (Tools):	
2.1 Overview	49
2.2 Application Function	50
2.3 Main Work	51
2.4 Mobile Content	53
2.5 Website Content	58
Chapter five (System implementation):	
5.1 Web Design	68
5.2 Mobile App Design	72
5.3 AI part.....	77
5.4 Code Snippets	88
Chapter six (Conclusion & Future work):	
6.1 Conclusion	93
6.2 Future Work	93
References	94

List of figures:

Figure 3.1: Use Case Diagram.....	39
Figure 3.2 Activity Diagram	40
Figure 3.3: Sequence Diagram	41
Figure 3.4: Class Diagram	42
Figure 3.5: DFD level 0	43
Figure 3.5: DFD level 1.....	44
Figure 3.5: DFD level 2.....	45
Figure 5.1: Home page	68
Figure5.2: Login/Register page.....	69
Figure 5.3: About Page	58
Figure 5.4: Skill Page	59
Figure 5.5: Home PageForMobileApplication.....	60
Figure 5.6: chat page	74
Figure 5.7: learning page (categories).....	76
Figure 5.8: Network architecture for the LSTM model.....	82
Figure 5.9: Sequential Long Short-Term Memory architecture.....	87
Figure 5.10: Speech to text.....	88
Figure 5.11: API Call in Webcam Functionality	89
Figure 5.12 Sending Frames to the Server.....	90
Figure 5.13mediapipe_detection.....	91
Figure 5.14 create_input_model_data.....	91

Chapter 1

Introduction

Introduction

In a world where talking and sharing ideas has become normal thing it's easy to forget that some folks face big hurdles in expressing themselves. People dealing with hearing and speech challenges often find it tough to jump into conversations or let their voices be heard. That's where the Unheard Voices project steps in, using the latest technology to make communication smoother for the deaf and mute community.

Unheard Voices is an innovative initiative that combines advanced speech recognition, sign language interpretation model, and text-to-speech synthesis to create a comprehensive communication platform. This platform empowers individuals with hearing and speech impairments to communicate effectively with those who may not understand sign language or are unfamiliar with their unique communication needs.

Key Features:

Speech Recognition: This feature allows a hearing and speaking individual to communicate with a deaf person through a mobile app. The spoken words are converted to

text, enabling the deaf person to read and understand the message.

Sign Language Interpretation: Unheard Voices includes a robust sign language recognition model that interprets and translates sign language gestures into text or spoken words. This feature ensures that individuals who primarily use sign language as their mode of communication can express themselves effortlessly.

Text-to-Speech Synthesis: The platform integrates advanced text-to-speech synthesis technology, allowing the generated text to be transformed into natural-sounding speech. This feature benefits users who prefer to receive information auditorily.

User-Friendly Interface: Unheard Voices boasts an intuitive and user-friendly interface designed with accessibility in mind. The interface includes conversation section between the talker and the listener.

Unheard Voices is committed to promoting accessibility and inclusivity by bridging the communication gap between individuals with hearing and speech impairments and those without. Through its innovative features, the project ensures that everyone has a voice and can participate in meaningful dialogues regardless of their communication abilities.

One of the core strengths of Unheard Voices is its ability to facilitate real-time communication between users. Whether it's converting spoken words to text, interpreting sign language gestures, or synthesizing text into speech, the platform enables seamless interactions that transcend traditional communication barriers.

Problem Definition

Individuals who are deaf or mute often encounter difficulties in expressing themselves and engaging in spontaneous conversations. The reliance on verbal communication in daily interactions can lead to feelings of isolation and frustration. Moreover, the limited understanding of sign language among the general population exacerbates these challenges, hindering effective communication between individuals with hearing and speech impairments and those without.

Educational Barriers:

In educational environments, the deaf and mute community faces additional hurdles. Traditional teaching methods heavily rely on verbal communication, making it challenging for students with hearing impairments to fully grasp instructional content. Limited access to sign language interpretation services and insufficient integration of assistive technologies further restrict educational opportunities for this community. As a result, achieving academic success and participating actively in classroom discussions become formidable tasks.

Communication Barriers in the Workplace:

The workplace often heavily relies on spoken communication, team collaboration, and verbal instructions. This creates a challenging environment for deaf and mute employees who may face difficulties in fully understanding verbal instructions, participating in spontaneous discussions, or engaging in casual workplace conversations. The lack of awareness and understanding of sign language among colleagues and employers exacerbates these challenges, isolating individuals with

hearing and speech impairments from the communicative dynamics of the workplace.

Limited Professional Development Opportunities:

Once employed, deaf and mute individuals often face limitations in accessing professional development opportunities. Workshops, training sessions, and team-building activities may not be inclusive or adapted to cater to diverse communication needs. This lack of accessibility can impede career growth and limit the potential for advancement within the organization.

Social Isolation and Stigma:

The communication barriers faced by the deaf and mute community can lead to social isolation and stigma. The inability to engage in spontaneous conversations and the lack of understanding from others can result in feelings of exclusion and marginalization. This isolation can have profound psychological impacts, contributing to low self-esteem, depression, and a sense of being misunderstood or undervalued by society.

Limitations in Accessing Information and Entertainment:

In today's information-driven society, the deaf and mute community faces challenges in accessing information and entertainment. Websites, videos, and other digital content may not be optimized for accessibility, making it difficult for individuals with hearing and speech impairments to fully engage with the content. This lack of accessibility can limit their ability to stay informed, learn new skills, and enjoy entertainment, further contributing to the digital divide.

Solution Approach

Solution Approach:

Dataset Collection:

Gather a diverse and extensive dataset of sign language videos paired with corresponding text transcripts. Ensure the dataset covers a wide range of sign language gestures, expressions, and variations to train a robust machine learning model effectively.

Machine Learning Model Training:

Utilize the collected dataset to train a machine learning model capable of recognizing and translating sign language gestures accurately.

Implement deep learning techniques, such as convolutional neural networks (CNNs) or recurrent neural networks (RNNs), to capture the nuances of sign language.

Mobile App Development:

Develop a user-friendly mobile app interface that caters to both deaf and hearing users. Design the app layout to be intuitive, accessible, and inclusive, with clear navigation and visual cues for seamless interaction.

Integration of Machine Learning Model:

Integrate the trained machine learning model into the mobile app to enable real-time translation of speech to sign language and vice versa.

Implement efficient algorithms for quick processing and accurate interpretation of gestures and spoken words.

User Testing and Feedback:

Conduct thorough testing of the app with deaf users to evaluate its accuracy, usability, and overall user experience. Gather feedback from

users to identify areas for improvement and refinement, ensuring that the app meets the needs and preferences of its target audience effectively.

By following this solution approach, the mobile app can serve as a valuable tool for facilitating communication between deaf and hearing individuals, promoting inclusivity, accessibility, and seamless interaction through the integration of machine learning technology.

Project Objectives:

1-Enhanced Communication: Provide a comprehensive solution that facilitates effective communication for deaf and mute individuals with the other natural talker including daily interactions, educational environments, and professional contexts.

2-Accuracy of translating sign language into text so that the speaker can read it or listen to the text clearly and understand the intent, Fast and Real-time translation into text without any delay and with high accuracy.

3-Teaching normal people and deaf people sign language so that they can communicate with each other more easily and comfortably.

4- Enable students who suffer from hearing loss or inability to speak to continue their lives alongside regular school students and not have to attend special schools.

Stakeholders

Now who will benefit from this project? individuals, groups, or entities that have an interest, involvement, or influence?

Deaf Individuals:

Anyone who has the ability to read and write but have lacks the sense of hearing or has hearing impairments.

Mute Individuals:

He has a speech problem or cannot speak normally or cannot speak at all.

Hearing Individuals:

Those without hearing or speech impairments who will be communicating with deaf and mute individuals through the platform,
Anyone interested in sign language and want to learn it to communicate easily with deaf and mute people.

Educational Institutions:

Schools, colleges, and universities that may adopt the technology to improve communication and accessibility for deaf and mute students.

Scope

There is a website and an application platforms specifically designed to address the unique needs of deaf and mute individuals. The project aims to facilitate effective communication in various aspects of life, including daily interactions, educational environments, and professional contexts.

Real-time Translation:

Emphasis will be placed on achieving high accuracy and real-time translation of sign language into text. The system will ensure that the translated text is presented promptly without delays, allowing for seamless and efficient communication.

Cross-platform Compatibility:

The communication platform will be designed to ensure cross-platform compatibility, allowing users to access the application seamlessly across various devices such as smartphones, tablets, and desktop computers, using flutter frame work to build application for any screen on multiple devices and platforms.

Constraints

Internet Connectivity Requirement:

The platform necessitates a reliable internet connection for real-time communication and data exchange. Users must have consistent access to the internet to utilize the features and functionalities of the communication platform effectively.

Cross-Platform Compatibility:

Users should have access to devices and platforms that support the communication application. This includes compatibility with various web browsers such as Chrome, operating systems including Windows, macOS, and mobile applications on iOS and Android platforms.

Browser Compatibility:

Users need to have a compatible web browser, such as Google Chrome, to access the web-based features of the communication platform. Compatibility with other widely used browsers may also be considered to enhance accessibility.

Mobile Application Availability:

The availability of a dedicated mobile application for both iOS and Android platforms is a constraint. Users must have access to smartphones or tablets running these operating systems to use the mobile application features.

Responsive Design for Various Devices:

The platform must feature a responsive design to accommodate different screen sizes and resolutions. This constraint ensures a consistent and user-friendly experience across a variety of devices.

Availability of Necessary Permissions:

Users must grant necessary permissions for the application to access device features like the camera and microphone. This is crucial for functionalities such as video calls and speech recognition.

Real-Time Communication Dependency:

The effectiveness of the platform relies on real-time communication capabilities. Delays in network latency or poor internet connectivity may impact the quality and immediacy of communication.

Advantages

The Unheard Voices application offers several advantages for deaf individuals in communicating with the world:

Enhanced Communication: The application provides a platform for deaf individuals to communicate effectively with others, bridging the gap between the Deaf and hearing worlds.

Inclusive Content: Through interviews, stories, and videos, the Unheard Voices application offers a diverse range of content that reflects the experiences of deafened adults and their families, promoting inclusivity and understanding.

Empowerment: By sharing stories and experiences, the application empowers deaf individuals, giving them a voice and a platform to express themselves, fostering a sense of empowerment and community.

Education and Awareness: The application raises awareness about the challenges faced by deaf individuals, educating the public and promoting a better understanding of deaf culture and experiences.

Support and Connection: Unheard Voices provides a supportive environment where deaf individuals can connect with others who share similar experiences, fostering a sense of community and belonging.

In summary, the Unheard Voices application plays a crucial role in facilitating communication, promoting inclusivity, empowering individuals, raising awareness, and fostering a sense of support and connection within the deaf community.

Chapter 2

Survey &

Historical data

Literature review

The history of applications and technologies designed to assist deaf people dates back centuries. In the 18th century, the Abbé de l'Épée founded the first state-supported school for deaf children in Paris, using signed language to teach written language[4]. This manual method contrasted with the oral method promoted by Samuel Heinicke in Germany, which emphasized speechreading and articulation[4].

In the 20th century, the development of assistive technologies accelerated. In the 1970s, real-time captioning was pioneered to provide subtitles for deaf and hard of hearing people watching television[1]. More recently, smartphone apps have emerged to help deaf people navigate public spaces and communicate with hearing individuals:

- Braci Sound Alert uses a smartphone's microphone to detect sounds like alarms and doorbells, providing visual and vibration alerts[2]
- Signly offers sign language translations of written information by pointing a phone at exhibits or documents[2]
- AVA provides live transcriptions of group conversations, displaying the name of each speaker[2]
- Rogervoice generates live transcriptions during phone calls[2]

While these technologies have greatly improved accessibility, challenges remain. Deaf people still face barriers accessing public announcements and communicating in noisy environments[1]. More research is needed to develop better icon designs and web accessibility guidelines for the deaf community[1].

Ava application

The Ava application was created by Ava, a company founded in 2016 with the mission of connecting the deaf and hard of hearing community with hearing individuals[5]. The app was developed over a two-year period to address the challenges deaf people face in group conversations[5].

AVA uses speech recognition technology to transcribe conversations in real-time, displaying the text on the user's smartphone screen[5]. By having multiple phones present and building voice profiles, the app can distinguish between speakers and provide a chat bubble-like transcription[5]. This allows deaf users to easily follow along and participate by typing and using text-to-speech[5].

The app aims to improve accessibility and inclusion for the deaf community, which comprises 5% of the world's population (300 million people)[5]. Group conversations in spoken language can be extremely difficult for deaf individuals to follow, as reading multiple people's lips simultaneously is nearly impossible[5].

AVA's launch video, created in collaboration with Prodigium Pictures, showcased real scenarios that beta testers had shared, resonating strongly with the deaf community[5]. The video went viral, garnering over 1 million views and 30,000 downloads within the first 3 days of the app's release[5]. This extraordinary performance highlighted the significant need for such an application.

By providing real-time transcriptions, AVA enables deaf and hard of hearing individuals to actively participate in group conversations, fostering better communication and social inclusion[5]. The app has the potential to greatly improve the quality of life for members of the deaf community by breaking down barriers and facilitating more seamless interactions with hearing people.

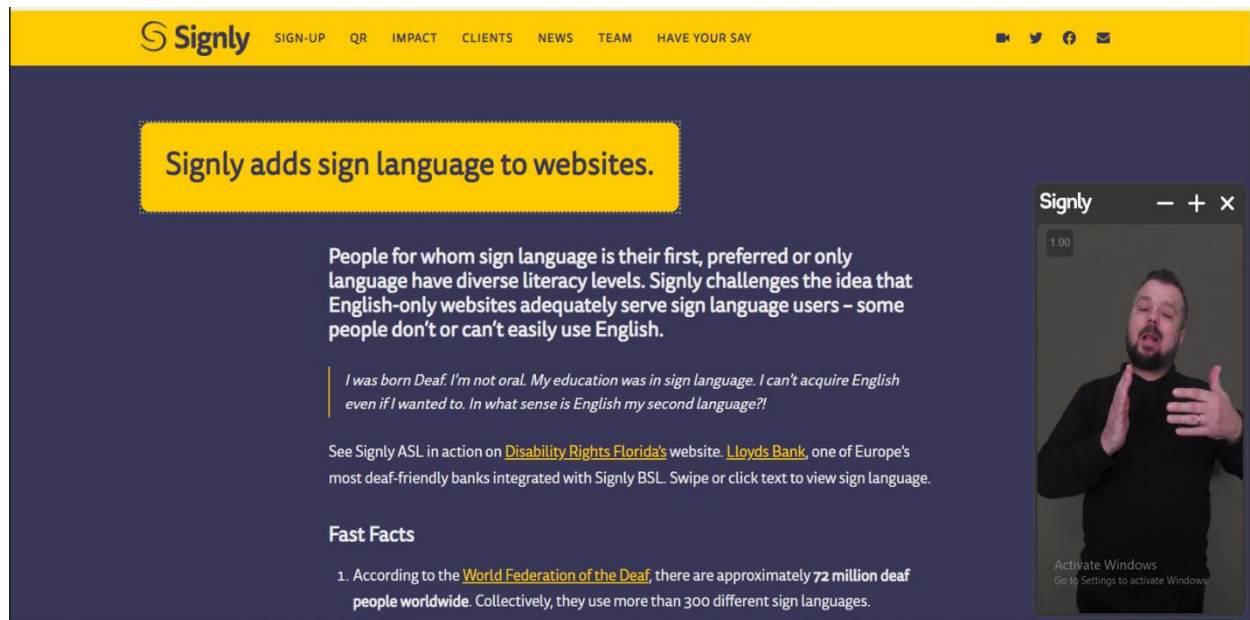


[6]

Signly

The Signly application was created in December 2015, The purpose of the application is to provide an innovative solution for individuals who are deaf or hard of hearing by offering sign language translations of written information.

Signly utilizes augmented reality technology to unlock accessibility for the deaf community, transforming the experience of visits to various places like museums, train stations, and banks. When users point their smartphones at information text or barcodes, Signly instantly displays pre-recorded sign language videos, enabling users to access content in a format that suits their needs. This application was developed to bridge the communication gap between written information and sign language, enhancing accessibility and understanding for individuals who rely on sign language for communication[7].



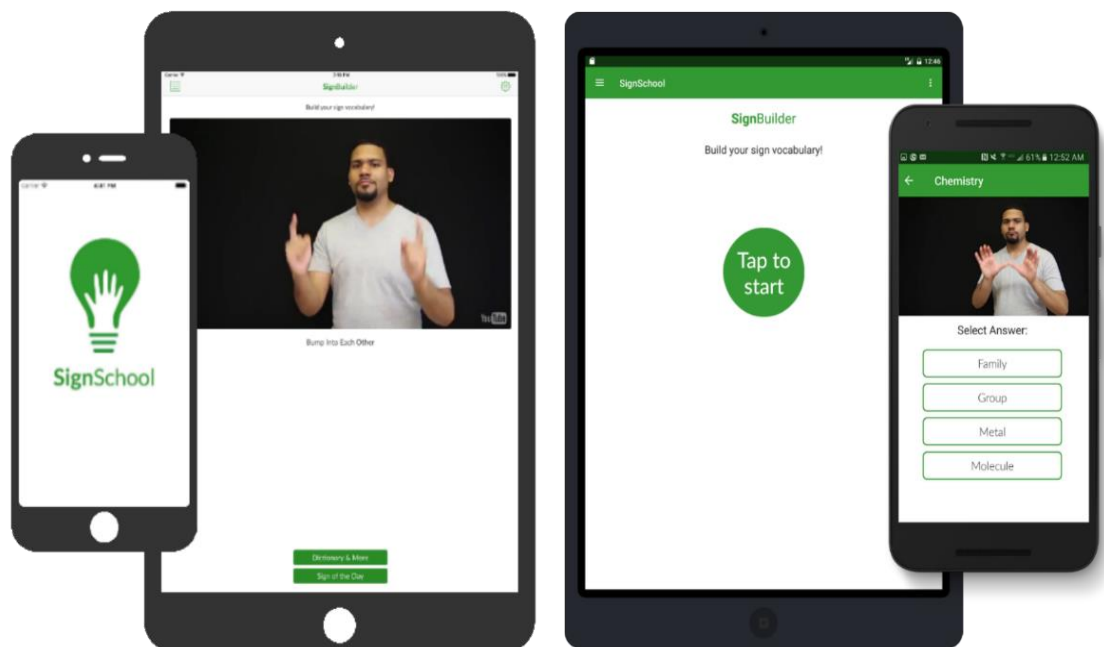
[8]

SignSchool

SignSchool was founded in 2014, SignSchool is a comprehensive app that offers lessons on American Sign Language (ASL) through interactive videos, quizzes, and games. Users can learn vocabulary, phrases, and grammar in ASL, with lessons categorized by topics such as numbers, colors, greetings, and more.

SignSchool provides a structured learning experience for beginners and advanced learners alike, allowing users to practice and improve their signing skills at their own pace. The app aims to make learning sign language fun, accessible, and engaging for users of all ages and levels of proficiency.[9]

[10]



Chapter 3

Software development life cycle SDLC

3.1 Project Development Methodology (phases) :

The waterfall approach is among the oldest project management techniques. It consists of sequential phases that map out every essential step of a project.

Each new step feeds into the next step in such a way that one can't begin without the previous phase's completion. When changes occur, they are costly because a phase's completion does not allow you to revisit it unless you restart the project from phase one.

1. Requirements and Planning

The requirements and planning phase of waterfall project management identifies what the project should do. The project manager tries to understand the project's requirements based on what the project sponsors need. This phase involves identifying and describing the project's risks, assumptions, dependencies, quality metrics, costs, and timeline.

2. Design

The design phase solidifies and documents all your decisions. In this case, you develop solutions that can solve the project's requirements.

The best way to do so is to note all the actions you'll take to deliver the project scope to execute them.

Design covers the project's schedule, budget, and objectives, and you can think of design as a blueprint or road map to the complete project.

3. Implementation

The implementation phase executes your project plan and design to produce the desired product. If your company develops software, you will spend this phase coding the software functionalities. Or, if you're managing a project at a construction company, you will construct a house in this phase.

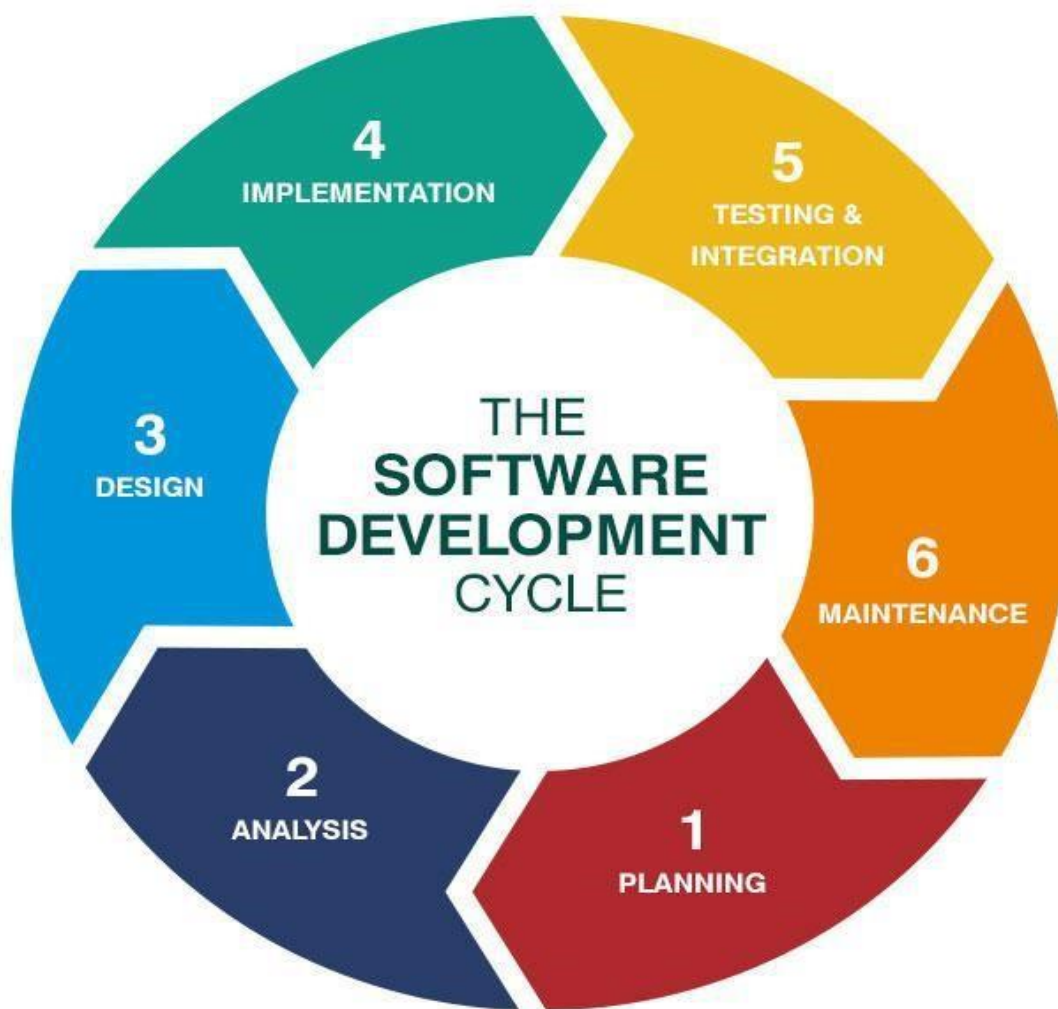
Implementation takes up a significant portion of waterfall project management. Everything that happens during this phase should be carefully documented.

4. Verification/Testing

Testing verifies that the product developed in the implementation phase fulfills the entire project's requirements. If this is not the case, the project team must review the project from phase one to identify what went wrong. The testing phase uses various quality metrics and customer satisfaction to measure the project's success.

5. Maintenance

The maintenance phase extends beyond the five stages of project management into the project's lifetime. This phase involves making minor modifications to improve the product developed during implementation and performing other routine maintenance tasks. It's also a phase to identify any errors you might have missed during the testing



3.2 Planning phase:

Define clear objectives for the project, including:

Developing a user-friendly mobile application interface.

Integrating interactive features for sign language learning.

Enabling real-time communication through text, voice, and video.

Implementing camera functionality for gesture recognition and feedback.

Providing login and registration functionalities for personalized user experiences.

3. Target Audience:

Identify the target audience for the application, including:

Individuals with hearing impairments (deaf and mute).

Normal users interested in learning sign language.

Educators, caregivers, and sign language enthusiasts.

4. Features and Functionalities:

Sign Language Learning Categories:

Develop structured learning categories covering basic to advanced sign language vocabulary and topics.

Interactive Learning Model:

Incorporate interactive lessons, videos, quizzes, and exercises to engage users in active learning experiences.

Real-Time Communication:

Enable text, voice, and video communication between users for real-time practice and feedback.

Camera Integration:

Implement camera functionality for gesture recognition, allowing users to practice sign language gestures and receive real-time feedback.

Voice Recording and Conversion to GIF:

Enable users to record voice and convert it into GIF sequences, facilitating expressive communication.

Camera Conversion to Voice:

Develop functionality to translate sign language gestures captured by the camera into voice output for seamless communication.

5. User Authentication:

Implement login and registration functionalities to personalize user experiences, track progress, and save preferences.

3.3 Analysis:

3.3.1 Requirements Analysis:

Requirements analysis, also known as requirements engineering, is a crucial phase in the development process of any new product. It is a systematic approach used to determine the needs and expectations of stakeholders and end-users, ensuring that the final product meets their requirements effectively. The requirements analysis process involves frequent communication and collaboration with stakeholders to define expectations, resolve conflicts, and document all key requirements accurately.

3.3.2 System Requirements:

Hardware:

Server:

A high-quality server is required to host the website and support the mobile application's backend operations. The server should have sufficient processing power, memory, and storage capacity to handle user requests efficiently.

Software:

PC Requirements:

Operating System: PCs should be running Windows 10 or higher to support the web application. For optimal performance in the Android Studio development environment, higher specifications are recommended.

Mobile Application Development:

Development Platform:

The mobile application is developed using Flutter, a cross-platform framework for building native mobile applications. Flutter provides a fast development environment and allows for seamless integration with Android and iOS platforms.

Web Application Development:

Frontend Technologies:

The web application's interface is designed using HTML, CSS, and JavaScript. These technologies provide flexibility and interactivity, allowing for the creation of dynamic and visually appealing user interfaces.

3.3.3 Software Requirements Specification :

1. Functional Requirements:

Sign Language Learning Categories:

Define categories covering various topics such as alphabet, numbers, greetings, emotions, daily activities, etc.

Interactive Learning Model:

Specify interactive elements such as videos, quizzes, exercises, and games to engage users in learning.

Real-Time Communication:

Enable text, voice, and video communication between users for practice sessions and feedback.

Camera Integration:

Implement camera functionality for gesture recognition and real-time feedback on sign language gestures.

Voice Recording and Conversion:

Allow users to record voice and convert it into GIF sequences for expressive communication.

Camera to Voice Conversion:

Develop functionality to translate sign language gestures captured by the camera into voice output.

User Authentication:

Implement login and registration functionalities for personalized user experiences and progress tracking.

2. Non-Functional Requirements:

Performance:

Ensure fast response times and smooth user experience, even with high concurrent usage.

Accessibility:

Design the app interface to be accessible and intuitive for users with varying levels of hearing impairment.

Security:

Implement robust security measures to protect user data and ensure privacy.

Compatibility:

Ensure compatibility with a wide range of mobile devices and operating systems (iOS, Android).

Scalability:

Design the application architecture to scale easily as the user base grows.

Usability:

Conduct usability testing to ensure the app is easy to navigate and understand for all users.

3.3.4 System Users:

In the system we have three main users:

- **Deaf and Mute Individuals:**

Primary users of the sign language learning application. They use the application to learn sign language, practice gestures, and communicate with others.

- **Normal Individuals:**

Users without hearing impairments who are interested in learning sign language. They use the application to improve their communication skills and interact with deaf and mute individuals.

3.4.4 Use Case Diagram.

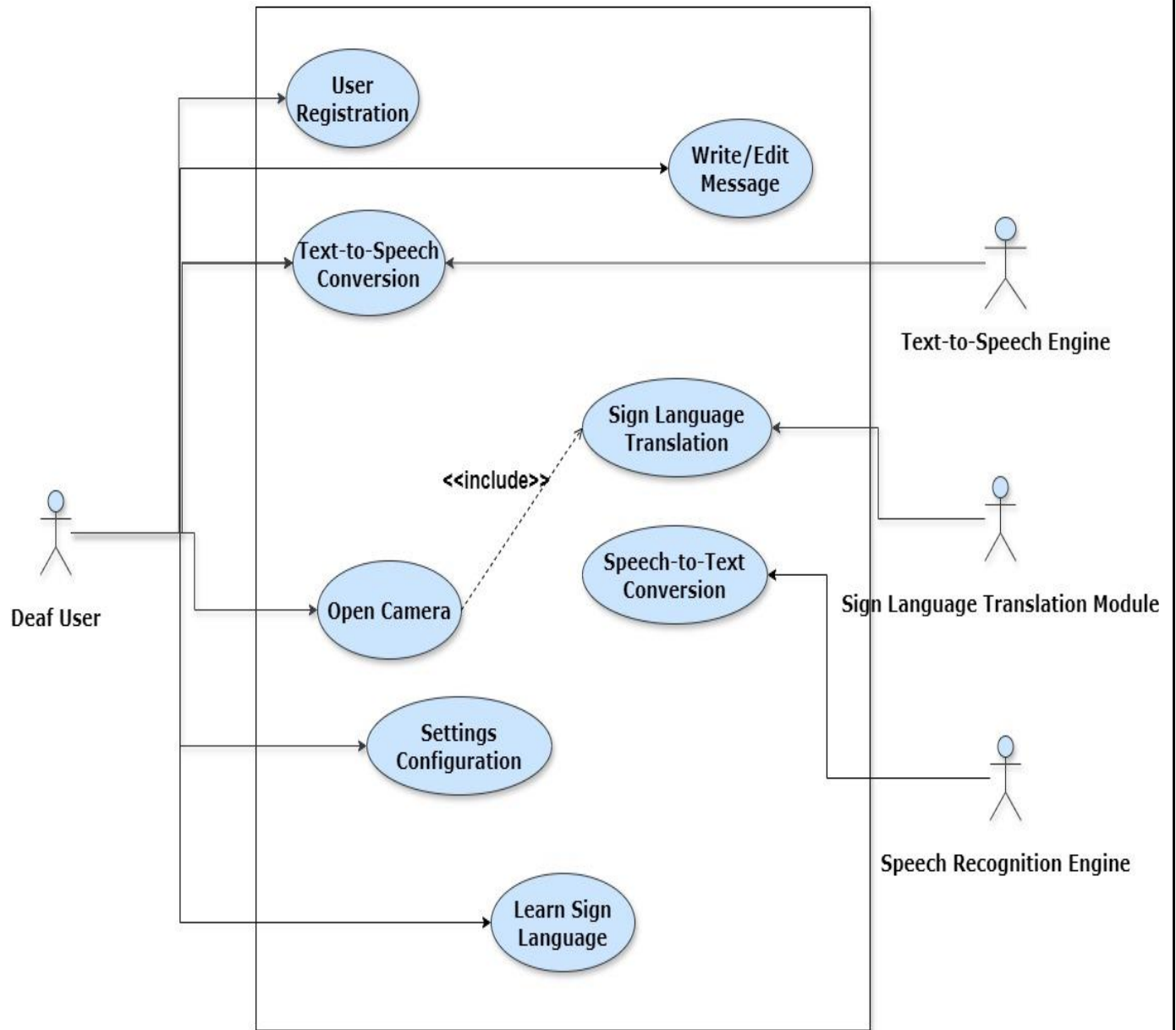


Figure 3.1: Use Case Diagram

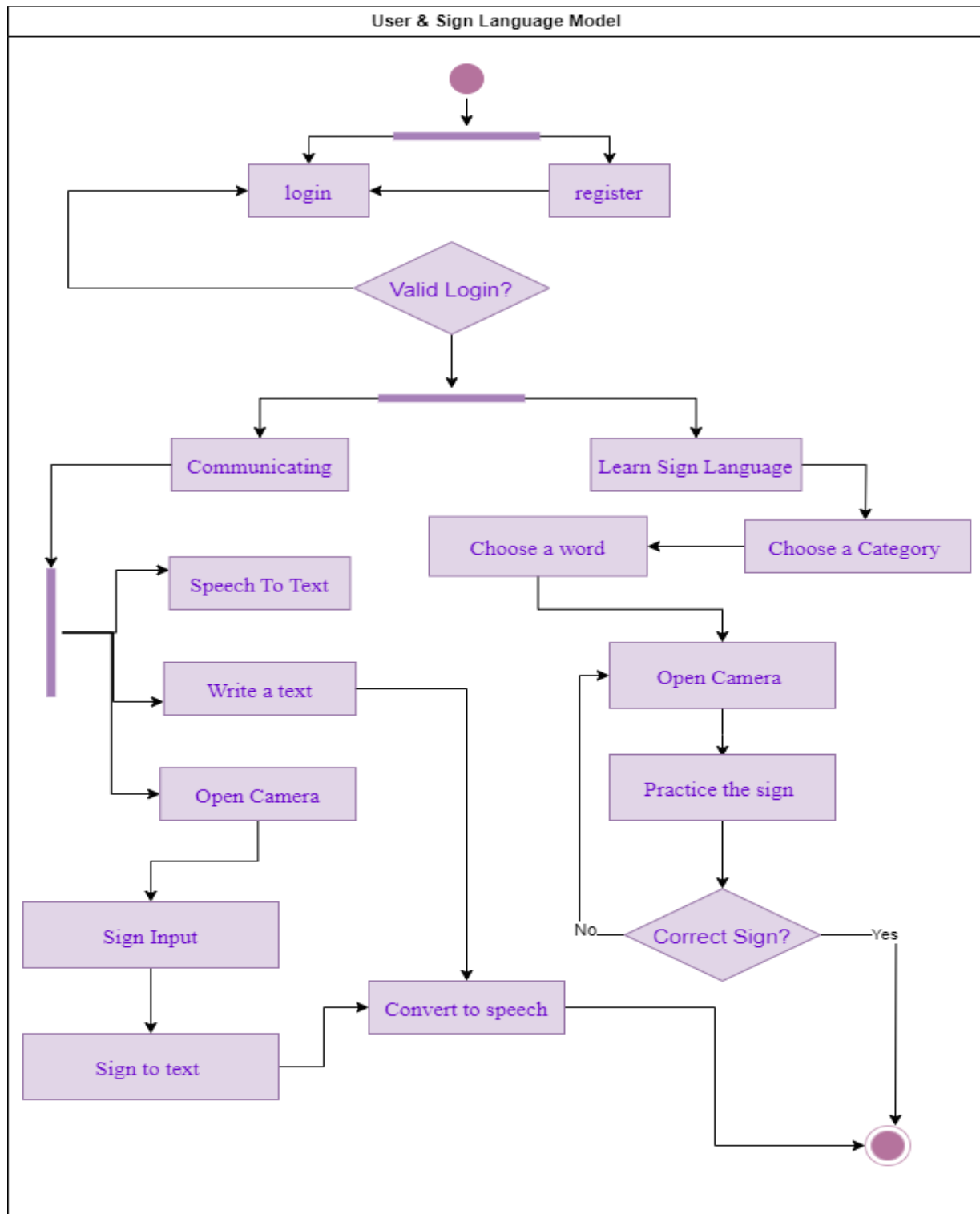


Figure 3.2: Activity Diagram

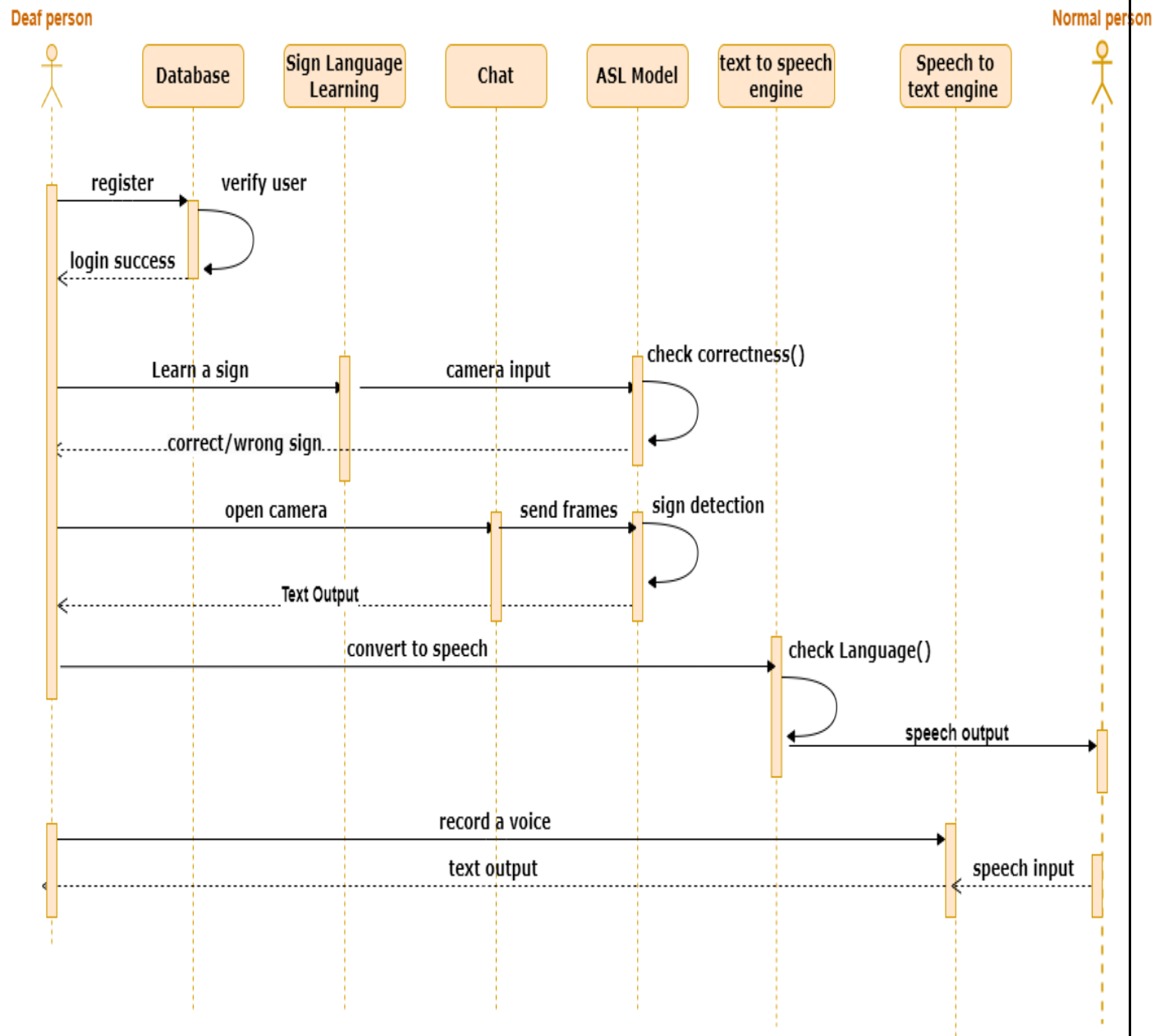


Figure 3.3: Sequence Diagram

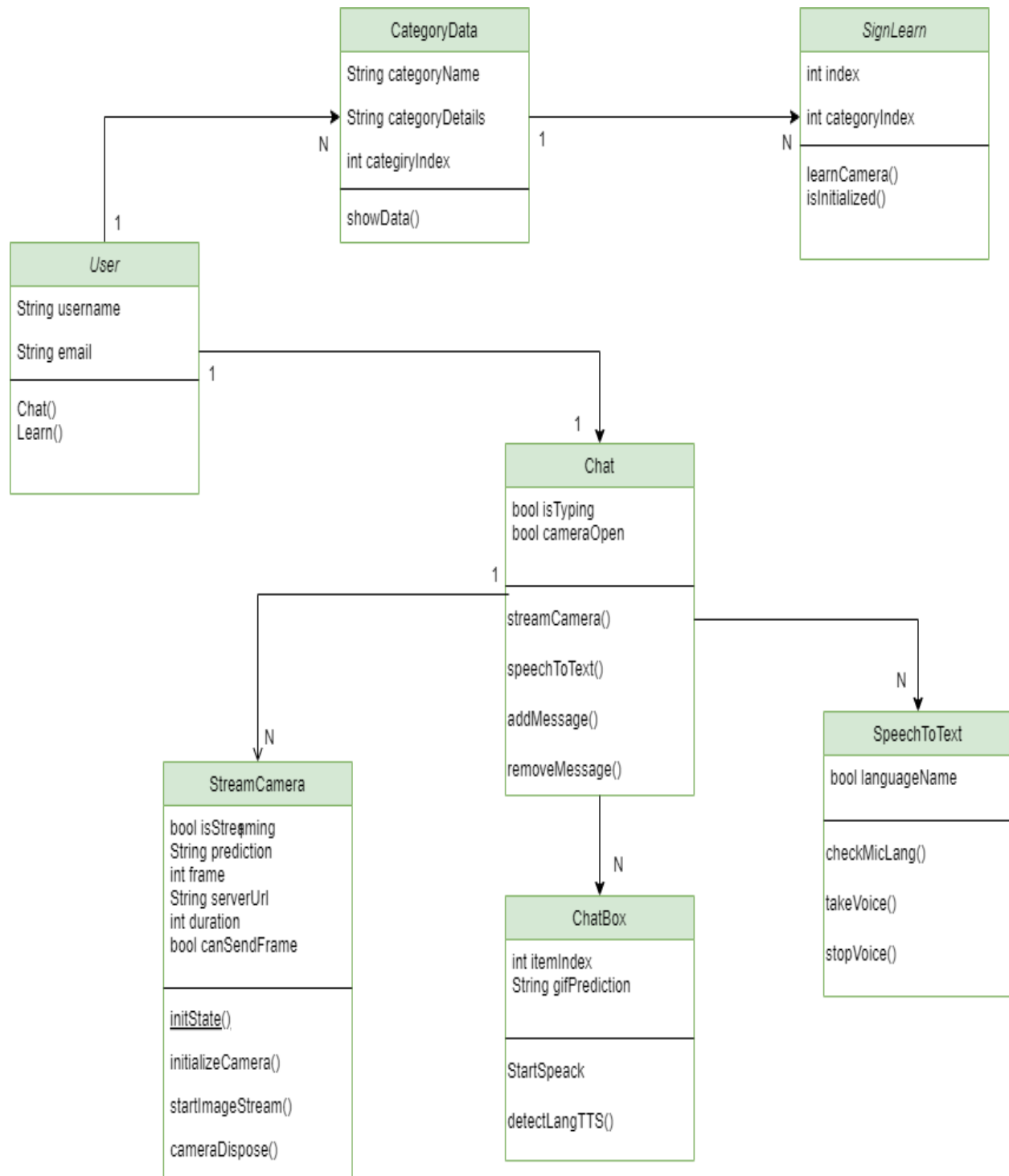


Figure 3.4: Class Diagram

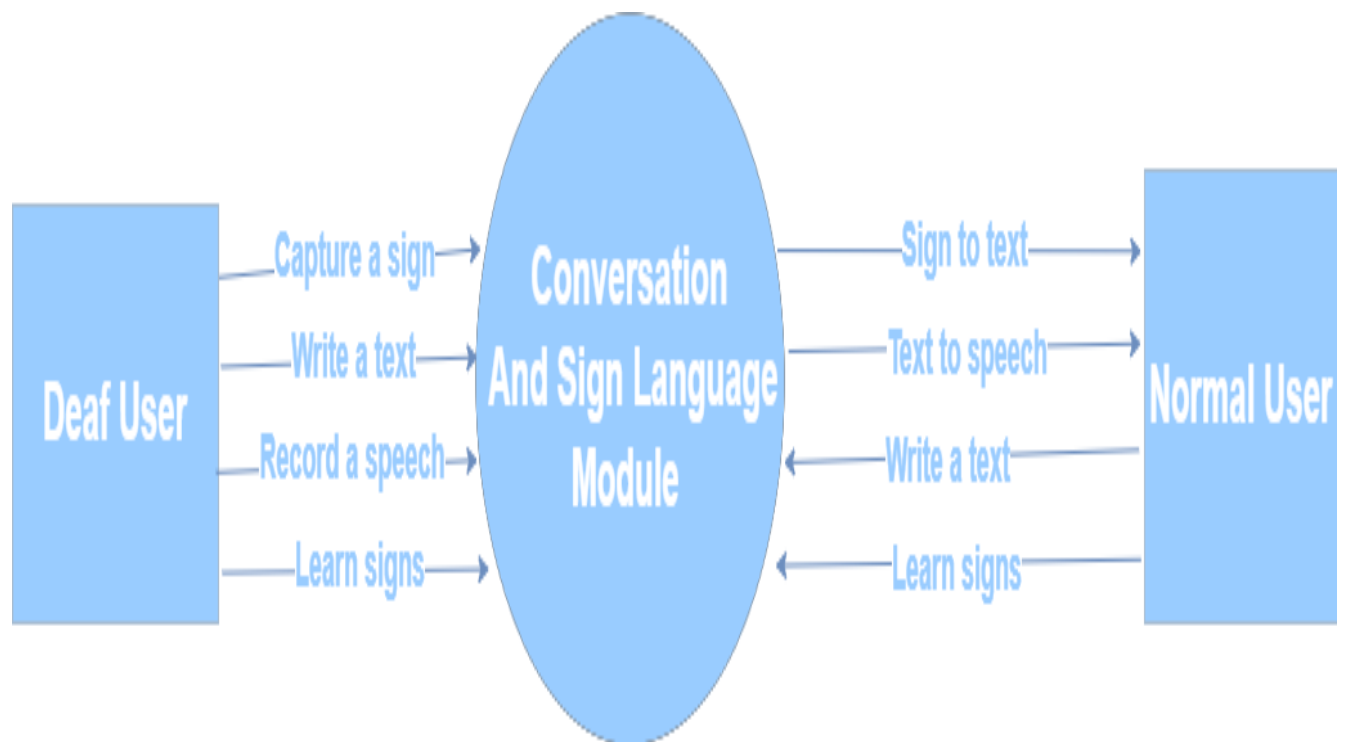


Figure 3.5: DFD level 0 Diagram

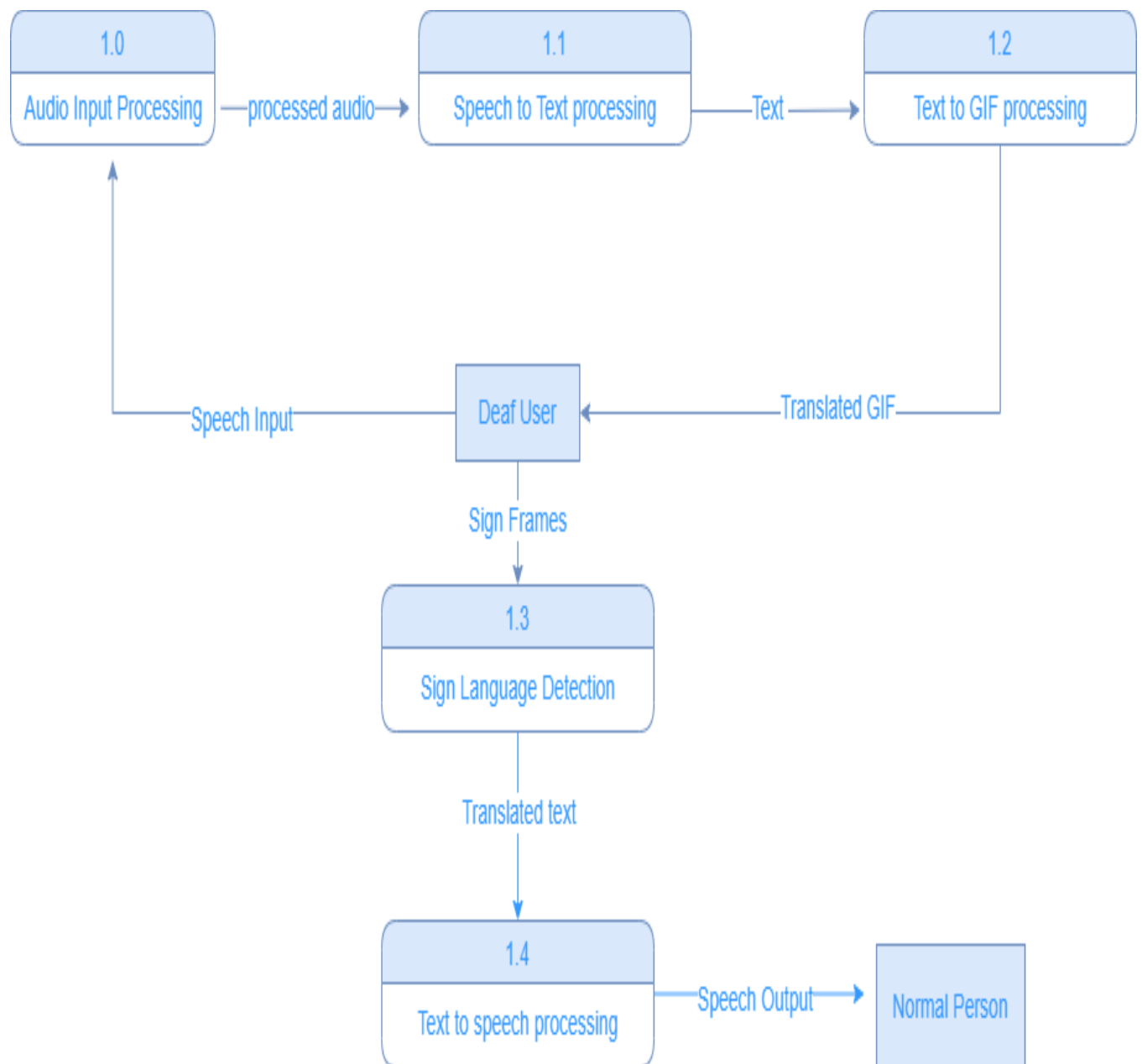


Figure 3.6: DFD level 1 Diagram

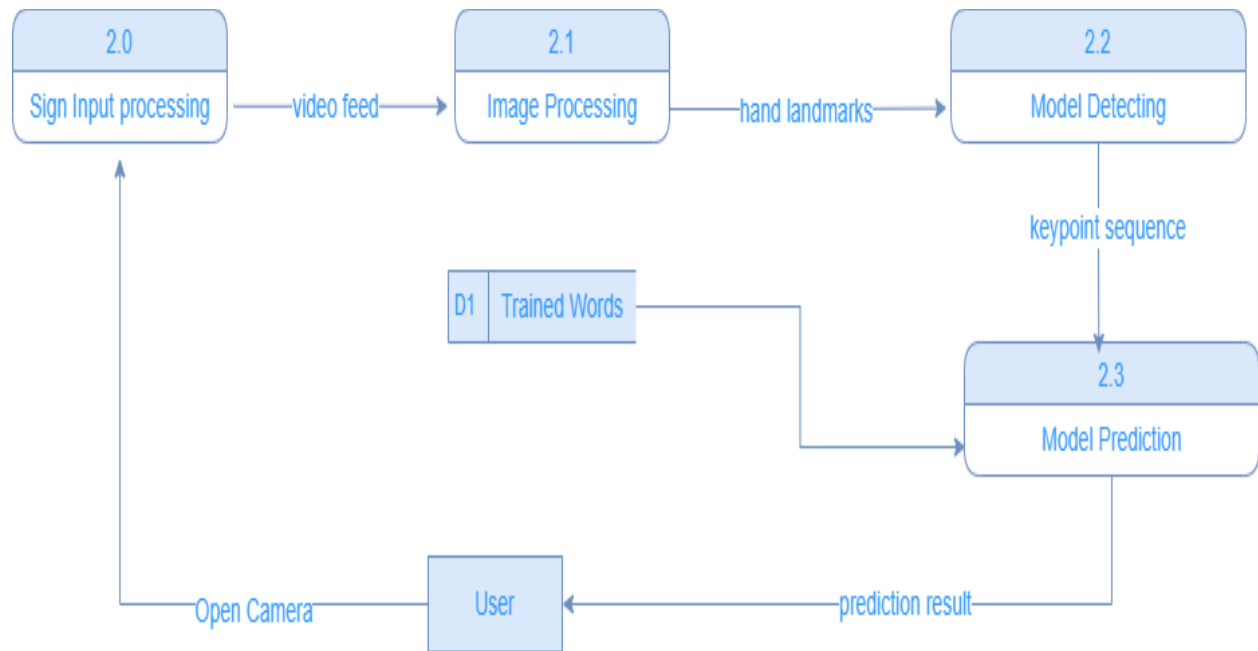


Figure 3.7: DFD level 2 Diagram

3.3.5 Tools:

- **For Web design (Front-end) using Visual Studio Code:**

- 1- HTML Language (Hyper Text Markup Language).
- 2- CSS (Styling Language).
- 3- Java Script (Scripting Language).

- **For Mobile Application using Android Studio:**

- 1- Flutter.
- 2- API 's

- **For Ai Machine learning using Kaggle**

Python

Machine learning

Deep learning

3.4 System Design :

The purpose of Design phase is to plan a solution for a problem specified by the requirements. System design aims to identify the modules that should be exist in the system,

the specification of those modules and how they interact with each other to produce the results. The goal of the design process is to produce a model that can be used later to build that system. The produced model is called design of the system.

System design is the process of defining the architecture, components, modules, interfaces and data for a system to satisfy specified requirements. Normally, the design proceeds in two stages:

- Physical design is a graphical representation of a system showing the system's internal and external entities and the flow of data into and out of these entities. An internal entity is an entity within the system that transforms data.

To represent the physical design of the system, we use diagrams like use case diagrams, etc.

Chapter 4 Tools

4.1 Overview:

Product Perspective:

The Unheard Voices application is designed to cater to the diverse needs of both the deaf and mute community and normal individuals, providing a platform for communication, learning, and inclusivity. Here's a breakdown of the application's components:

Deaf and Mute Users:

For deaf and mute users, the application serves as a vital tool for communication. It includes features such as conversation functionality, allowing users to engage in meaningful discussions using sign language. Through innovative camera integration, sign language gestures are translated into voice output, enabling seamless communication with others.

Normal Users:

Normal individuals utilize the application for learning sign language. It offers a wide range of learning categories tailored to different skill levels and interests. Users can explore categories such as alphabets, numbers, greetings, and daily activities, facilitating their journey to mastering sign language.

4.2 Application Functions:

The system/application caters to the diverse needs of both the deaf and mute community and normal individuals, fostering communication, learning, and inclusivity. Here are the key functionalities:

User Authentication:

To access all features, users, whether deaf and mute or normal, must sign up/sign in. This ensures personalized access and data security.

Learning in Categories:

Users can explore a variety of categories tailored to their learning preferences. Whether it's alphabets, numbers, greetings, or daily activities, individuals can choose the categories they wish to learn from.

Camera Integration for Deaf and Mute Users:

A standout feature enables deaf and mute users to open the camera and convert sign language gestures into voice output. This innovative functionality enhances communication by translating sign language into spoken language.

Voice to GIF Conversion for Deaf and Mute Users:

Deaf and mute users can also benefit from voice to GIF conversion during conversations. This feature allows users to communicate seamlessly by converting voice messages from others into GIFs, ensuring effective communication.

Learning and Communication for Normal Users:

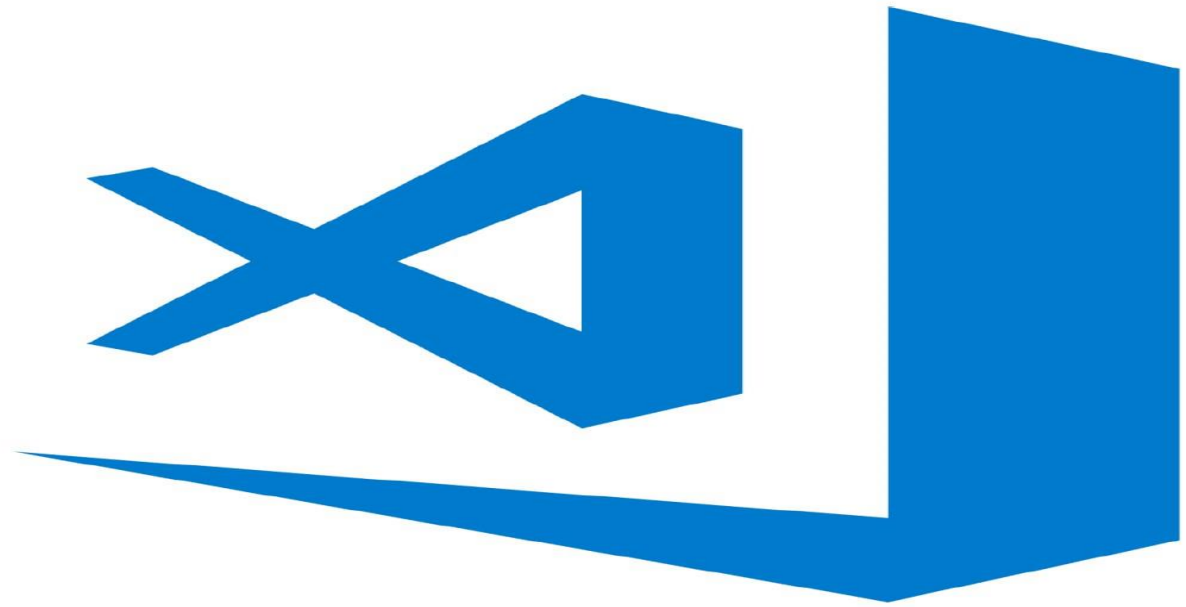
Normal individuals can utilize the camera to learn sign language words in various categories. Additionally, they can communicate with others through the application, fostering understanding and inclusivity.

4.3 Main Work :

Visual Studio Code : **Visual Studio Code**, also commonly referred to as **VS Code**, is a source-code editor developed by Microsoft for Windows, Linux, macOS and web browsers. Features include support for debugging, syntax highlighting, intelligent code completion, snippets, code refactoring, and embedded version control with Git. Users can change the theme, keyboard shortcuts, preferences, and install extensions that add functionality.

In the Stack Overflow 2023 Developer Survey, Visual Studio Code was ranked the most popular developer environment tool among 86,544 respondents, with 73.71% reporting that they use it.

Visual Studio Code is a source-code editor that can be used with a variety of programming languages, including C, C#, C++, Fortran, Go, Java, JavaScript, Node.js, Python, Rust, and Julia. Visual Studio Code employs the same editor component (codenamed "Monaco") used in Azure DevOps (formerly called "Visual Studio Online" and "Visual Studio Team Services").



Android Studio : is the official^[7] integrated development environment (IDE) for Google's Android operating system, built on JetBrains' IntelliJ IDEA software and designed specifically for Android development.^[8] It is available for download on Windows, macOS and Linux based operating systems.^[9] It is a replacement for the Eclipse Android Development Tools (E-ADT) as the primary IDE for native Android application development. Android Studio is licensed under the Apache license but it ships with some SDK updates that are under a non-free license, making it not open source.^[10]

Android Studio was announced on May 16, 2013, at the Google I/O conference. It was in early access preview stage starting from version 0.1 in May 2013, then entered beta stage starting from version 0.8 which was released in June 2014.^[11] The first stable build was released in December 2014, starting from version 1.0.^[12] At the end of 2015, Google dropped support for Eclipse ADT, making Android Studio the only officially supported IDE for Android development.



-
-

4.4 Mobile Content :

Flutter is an open-source UI software development kit created by Google. It can be used to develop cross platform applications from a single codebase for the web,^[4] Fuchsia, Android, iOS, Linux, macOS, and Windows.^[5] First described in 2015,^{[6][7]} Flutter was released in May 2017. Flutter is used internally by Google in apps such as Google Pay^[8] and Google Earth^[9] as well as by other software developers including ByteDance^[10] and Alibaba.^[11]

Flutter consists of both a UI language and a rendering engine. When a Flutter application is compiled, it ships with both the UI code and the rendering engine, which is about 4 MB compressed.^[12] This is in

contrast to many other UI frameworks that rely on a separate rendering engine and only ship the UI code, such as native Android apps which rely on the device-level Android SDK or HTML/JavaScript Web apps that rely on the user's HTML engine and JavaScript engine.^{[13][14]} Flutter's complete control of its rendering pipeline makes supporting multiple platforms simpler as it only needs the platform to support running native code such as via the Android Java Native Interface rather than support Flutter's UI model in its entirety.



What is Dart ?

Dart is a client-optimized language for developing fast apps on any platform. Its goal is to offer the most productive programming language for multi-platform development, paired with a flexible execution runtime platform for app frameworks.

Languages are defined by their *technical envelope*—the choices made during development that shape the capabilities and strengths of a language. Dart is designed for a technical envelope that is particularly suited to client development, prioritizing both development (sub-second stateful hot reload) and high-quality production experiences across a wide variety of compilation targets (web, mobile, and desktop).

Dart also forms the foundation of Flutter. Dart provides the language and runtimes that power Flutter apps, but Dart also supports many core developer tasks like formatting, analyzing, and testing code.

Dart: The language

The Dart language is type safe; it uses static type checking to ensure that a variable's value *always* matches the variable's static type. Sometimes, this is referred to as sound typing. Although types are mandatory, type annotations are optional because of type inference. The Dart typing system is also flexible, allowing the use of a dynamic type combined with runtime checks, which can be useful during experimentation or for code that needs to be especially dynamic.

Dart: The libraries

Dart has a rich set of core libraries, providing essentials for many everyday programming tasks:

- Built-in types, collections, and other core functionality for every Dart program (`dart:core`)
- Richer collection types such as queues, linked lists, hashmaps, and binary trees (`dart:collection`)
- Encoders and decoders for converting between different data representations, including JSON and UTF-8 (`dart:convert`)
- Mathematical constants and functions, and random number generation (`dart:math`)
- Support for asynchronous programming, with classes such as `Future` and `Stream` (`dart:async`)
- Lists that efficiently handle fixed-sized data (for example, unsigned 8-byte integers) and SIMD numeric types (`dart:typed_data`)
- File, socket, HTTP, and other I/O support for non-web applications (`dart:io`)
- Foreign function interfaces for interoperability with other code that presents a C-style interface (`dart:ffi`)
- Concurrent programming using *isolates*—independent workers that are similar to threads but don't share memory, communicating only through messages (`dart:isolate`)
- HTML elements and other resources for web-based applications that need to interact with the browser and the Document Object Model (DOM) (`dart:html`)

Beyond the core libraries, many APIs are provided through a comprehensive set of packages. The Dart team publishes many useful supplementary packages, such as these:

- `characters`
- `intl`
- `http`
- `crypto`
- `markdown`

Additionally, third-party publishers and the broader community publish thousands of packages, with support for features like these:

- [XML](#)
- [Windows integration](#)
- [SQLite](#)
- [compression](#)

To see a series of working examples featuring the Dart core libraries, read the [core library documentation](#). To find additional APIs, check out the [commonly used packages page](#).

Dart: The platforms

<#>

Dart's compiler technology lets you run code in different ways:

- **Native platform:** For apps targeting mobile and desktop devices, Dart includes both a Dart VM with just-in-time (JIT) compilation and an ahead-of-time (AOT) compiler for producing machine code.
- **Web platform:** For apps targeting the web, Dart can compile for development or production purposes. Its web compilers translate Dart into JavaScript or WebAssembly.

Dart has built-in [sound null safety](#). This means values can't be null unless you say they can be. With sound null safety, Dart can protect you from null exceptions at runtime through static code analysis. Unlike many other null-safe languages, when Dart determines that a variable is non-nullable, that variable can never be null. If you inspect your running code in the debugger, you see that non-nullability is retained at runtime; hence *sound* null safety.



Dart

4.5 Website Content :

-Front-End :

HyperText Markup Language (HTML) : is the standard markup language for documents designed to be displayed in a web browser. It defines the content and structure of web content. It is often assisted by technologies such as Cascading Style Sheets (CSS) and scripting languages such as JavaScript.

Web browsers receive HTML documents from a web server or from local storage and render the documents into multimedia web pages. HTML describes the structure of a web page semantically and originally included cues for its appearance.

HTML elements are the building blocks of HTML pages. With HTML constructs, images and other objects such as interactive forms may be embedded into the rendered page. HTML provides a means to create structured documents by denoting structural semantics for text such as headings, paragraphs, lists, links, quotes, and other items. HTML elements are delineated by *tags*, written using angle brackets. Tags such as `<img>` and `<input>` directly introduce content into the page. Other tags such as `<p>` and `</p>` surround and provide

information about document text and may include sub-element tags. Browsers do not display the HTML tags but use them to interpret the content of the page.

HTML can embed programs written in a scripting language such as JavaScript, which affects the behavior and content of web pages. The inclusion of CSS defines the look and layout of content. The World Wide Web Consortium (W3C), former maintainer of the HTML and current maintainer of the CSS standards, has encouraged the use of CSS over explicit presentational HTML since 1997.^[2] A form of HTML, known as HTML5, is used to display video and audio, primarily using the `<canvas>` element, together with JavaScript.

Cascading Style Sheets (CSS) : is a style sheet language used for specifying the presentation and styling of a document written in a markup language such as HTML or XML (including XML dialects such as SVG, MathML or XHTML). CSS is a cornerstone technology of the World Wide Web, alongside HTML and JavaScript.

CSS is designed to enable the separation of content and presentation, including layout, colors, and fonts. This separation can improve content accessibility;¹ provide more flexibility and control in the specification of presentation characteristics; enable multiple web pages to share formatting by specifying the relevant CSS in a separate .css file, which reduces complexity and repetition in the structural content; and enable the .css file to be cached to improve the page load speed between the pages that share the file and its formatting.

Separation of formatting and content also makes it feasible to present the same markup page in different styles for different rendering methods, such as on-screen, in print, by voice (via speech-based browser or screen reader), and on Braille-based tactile devices. CSS also has rules for alternate formatting if the content is accessed on a mobile device.

The name *cascading* comes from the specified priority scheme to determine which declaration applies if more than one declaration of a property match a particular element. This cascading priority scheme is predictable.

The CSS specifications are maintained by the World Wide Web Consortium (W3C). Internet media type (MIME type) text/css is registered for use with CSS by RFC 2318 (March 1998). The W3C operates a free CSS validation service for CSS documents.^[5]

In addition to HTML, other markup languages support the use of CSS including XHTML, plain XML, SVG, and XUL. CSS is also used in the GTK widget toolkit.



JavaScript : **JavaScript** (/ˈdʒɑːvəskript/), often abbreviated as **JS**, is a programming language and core technology of the Web, alongside HTML and CSS. 99% of websites use JavaScript on the client side for webpage behavior.^[10]

Web browsers have a dedicated JavaScript engine that executes the client code. These engines are also utilized in some servers and a variety of apps. The most popular runtime system for non-browser usage is Node.js.

JavaScript is a high-level, often just-in-time compiled language that conforms to the ECMAScript standard.^[11] It has dynamic typing, prototype-based object-orientation, and first-class functions. It is multi-paradigm, supporting event-driven, functional, and imperative programming styles. It has application programming interfaces (APIs) for working with text, dates, regular expressions, standard data structures, and the Document Object Model (DOM).

The ECMAScript standard does not include any input/output (I/O), such as networking, storage, or graphics facilities. In practice, the web browser or other runtime system provides JavaScript APIs for I/O.

Although Java and JavaScript are similar in name, syntax, and respective standard libraries, the two languages are distinct and differ greatly in design.

Trademark

"JavaScript" is a trademark of Oracle Corporation in the United States.^{[36][37]} The trademark was originally issued to Sun Microsystems on 6 May 1997, and was transferred to Oracle when they acquired Sun in 2009.^[38]

Website client-side usage

JavaScript is the dominant client-side scripting language of the Web, with 99% of all websites using it for this purpose.^[10] Scripts are

embedded in or included from HTML documents and interact with the DOM.

All major web browsers have a built-in JavaScript engine that executes the code on the user's device.

Examples of scripted behavior

- Loading new web page content without reloading the page, via Ajax or a WebSocket. For example, users of social media can send and receive messages without leaving the current page
- Web page animations, such as fading objects in and out, resizing, and moving them.
- Playing browser games.
- Controlling the playback of streaming media.
- Generating pop-up ads or alert boxes.
- Validating input values of a web form before the data is sent to a web server.
- Logging data about the user's behavior then sending it to a server. The website owner can use this data for analytics, ad tracking, and personalization.
- Redirecting a user to another page.
- Storing and retrieving data on the user's device, via the storage or IndexedDB standards.



- **Artificial intelligence (Ai) content :-**

The screenshot displays the Kaggle platform interface. On the left is a sidebar with navigation links: Home, Competitions, Datasets, Models, Code, Discussions, Learn, More, Your Work, and a 'VIEWED' section listing recent competitions. The main content area features a search bar at the top. Below it, a large banner for the 'Google - Isolated Sign Language Recognition' competition is shown, with a prize of \$100,000. The banner includes the Google logo, the number of teams (1,165), and the time since the competition started (6 months ago). Below the banner are tabs for Overview, Data, Code, Models, Discussion, Leaderboard, Rules, and Team. The 'Overview' tab is selected, showing a timeline from February 23, 2023, to May 2, 2023, with a 'Merger & Entry' point. To the right of the timeline, there are sections for 'Competition Host' (Google), 'Prizes & Awards' (\$100,000), and 'Participation' (1,340 Competitors, 1,165 Teams, 20,210 Entries). A 'Tags' section is also visible at the bottom right. A footer at the very bottom states: 'Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.' with links for 'Learn more.' and 'Ok, Got it.'

Kaggle is a data science competition platform and online community of data scientists and machine learning practitioners under Google LLC. Kaggle enables users to find and publish datasets, explore and build models in a web-based data science environment, work with other data scientists and machine learning engineers, and enter competitions to solve data science challenges.

Kaggle has implemented a progression system to recognize and reward users based on their contributions and achievements within the platform. This system consists of five tiers: Novice, Contributor, Expert, Master, and Grandmaster. Each tier is achieved by meeting specific criteria in competitions, datasets, kernels (code-sharing), and discussions.

The highest and most prestigious tier, Kaggle Grandmaster, is awarded to users who demonstrate exceptional skills in data science and machine learning. Achieving this status is extremely challenging. As of April 4, 2023, out of 12 million Kaggle users, only 2,331 (about 1 out of every 5500 users) have reached the Master level.

Among these Masters, only 472 (approximately 1 out of every 5 Masters) have achieved the coveted Kaggle Grandmaster status.

The other tiers in the progression system include:

- 13 thousand Experts
- 200 thousand Contributors
- 12 million Novices.

The progression system serves to motivate users to continuously improve their skills and contribute to the Kaggle community.

Python (programming language)

Python is a high-level, general-purpose programming language. Its design philosophy emphasizes code readability with the use of significant indentation.

Python is dynamically typed and garbage-collected. It supports multiple programming paradigms, including structured (particularly procedural), object-oriented and functional programming. It is often described as a

"batteries included" language due to its comprehensive standard library.

Guido van Rossum began working on Python in the late 1980s as a successor to the ABC programming language and first released it in 1991 as Python 0.9.0. Python 2.0 was released in 2000. Python 3.0, released in 2008, was a major revision not completely backward-compatible with earlier versions. Python 2.7.18, released in 2020, was the last release of Python 2.

Python consistently ranks as one of the most popular programming languages, and has gained widespread use in the machine learning community

Python is a multi-paradigm programming language. Object-oriented programming and structured programming are fully supported, and many of their features support functional programming and aspect-oriented

programming (including metaprogramming and metaobjects) Many other paradigms are supported via extensions, including design by contract and logic programming. Python uses dynamic typing and a combination of reference counting and a cycle-detecting garbage collector for memory management. It uses dynamic name resolution (late binding), which binds method and variable names during program execution



Chapter 5

System

Implementation

5.1 Web Design :-

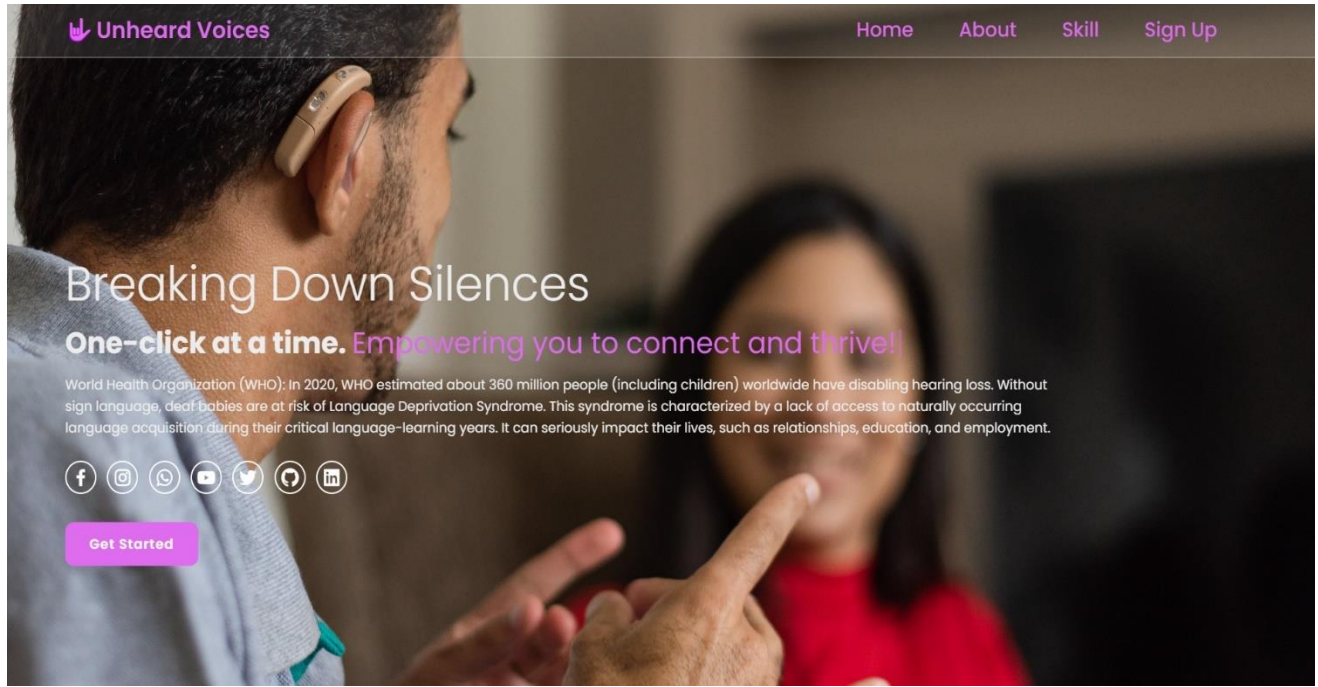


Figure 5.1 Home page

The home page acts as the central hub for the Unheard Voices Program website, offering a comprehensive overview of the available fields and functionalities, including sign language learning and communication features for both

Users can browse through the listed fields to explore opportunities within the Unheard Voices Program.

Normal users can utilize the sign language learning feature to acquire basic sign language skills, enhancing communication with the deaf and hard of hearing community.

Both normal users and individuals with hearing impairments can utilize the communication tools

provided on the website to interact and engage in meaningful conversations.

normal users and those with hearing impairment

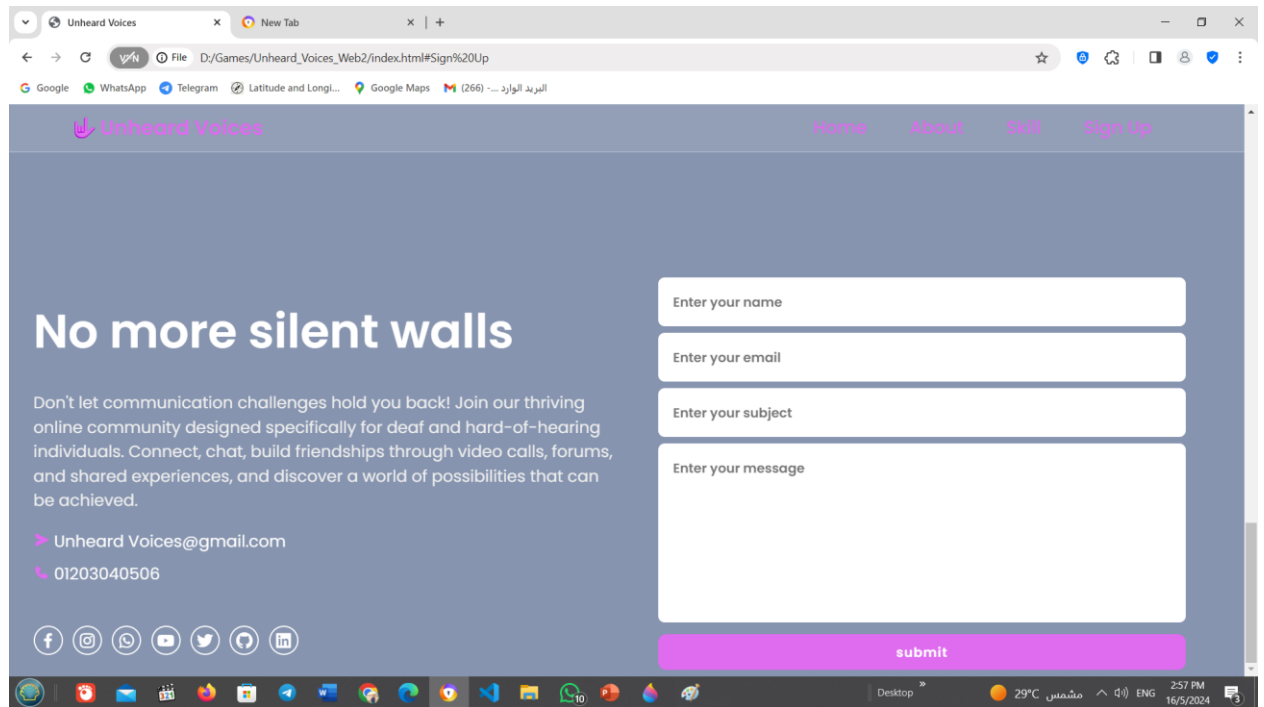
The image is a screenshot of a web browser displaying the 'Unheard Voices' login/register page. The browser's address bar shows the file path 'D:/Games/Unheard_Voices_Web2/index.html#Sign%20Up'. The website has a purple header with the logo 'Unheard Voices' and navigation links for 'Home', 'About', 'Skill', and 'Sign Up'. The main content area has a dark blue background. On the left, the heading 'No more silent walls' is followed by a paragraph: 'Don't let communication challenges hold you back! Join our thriving online community designed specifically for deaf and hard-of-hearing individuals. Connect, chat, build friendships through video calls, forums, and shared experiences, and discover a world of possibilities that can be achieved.' Below this, contact information is listed: 'Unheard Voices@gmail.com' and '01203040506'. A row of social media icons (Facebook, Instagram, WhatsApp, YouTube, Twitter, Telegram, LinkedIn) is at the bottom left. On the right, there is a registration form with four input fields: 'Enter your name', 'Enter your email', 'Enter your subject', and 'Enter your message'. A purple 'submit' button is at the bottom right of the form. The browser's taskbar at the bottom shows various application icons and system information like 'Desktop', '29°C', and the date '16/5/2024'.

Figure 5.2 Login/Register page

The Login/Register page serves as a gateway to the Unheard Voices Program website, offering seamless access and engagement opportunities for both normal users and individuals with hearing impairments. By prioritizing accessibility and inclusivity, the page aims to foster a supportive online community where users can connect, learn, and advocate for deaf and hard of hearing rights. Through its integration of sign language support and user-friendly authentication mechanisms, the Login/Register page exemplifies the Unheard Voices Program's commitment to promoting communication and understanding among individuals of all backgrounds.

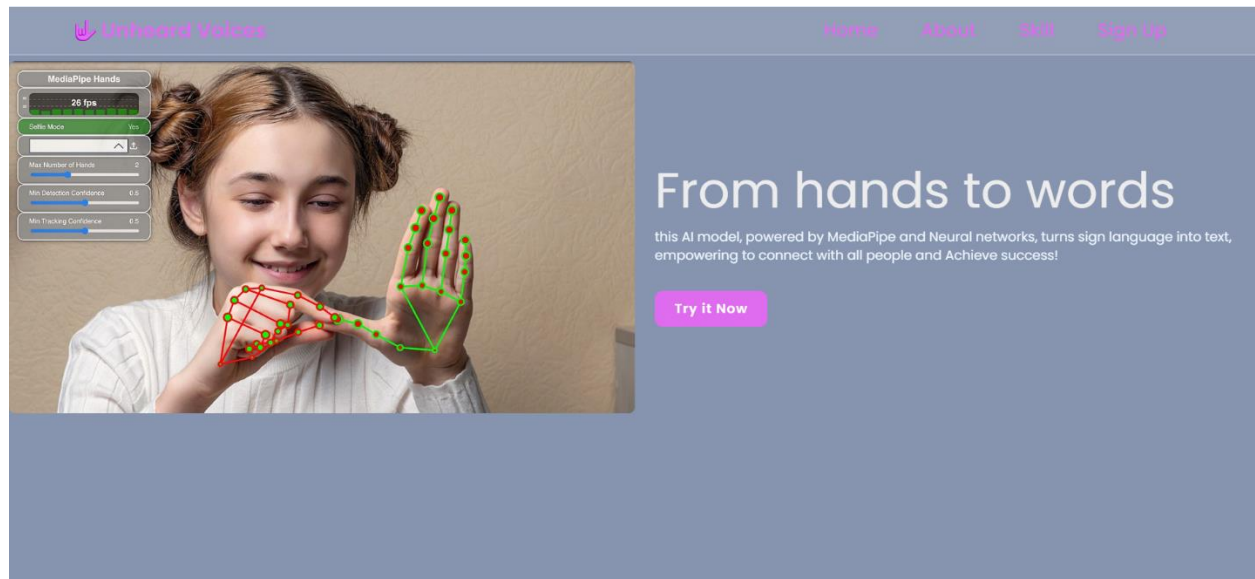


Figure 5.3 About page

One of the key functionalities of the About page is its encouragement for users to "Try It." Through engaging prompts and invitations, users are encouraged to initiate conversations using sign language, whether they are normal individuals or those with hearing impairments.

Through clear prompts and invitations, users are encouraged to take the next step in their sign language learning journey by initiating conversations. Whether they are normal individuals or those with hearing impairments, users are empowered to use sign language in real-life scenarios, fostering communication and understanding.

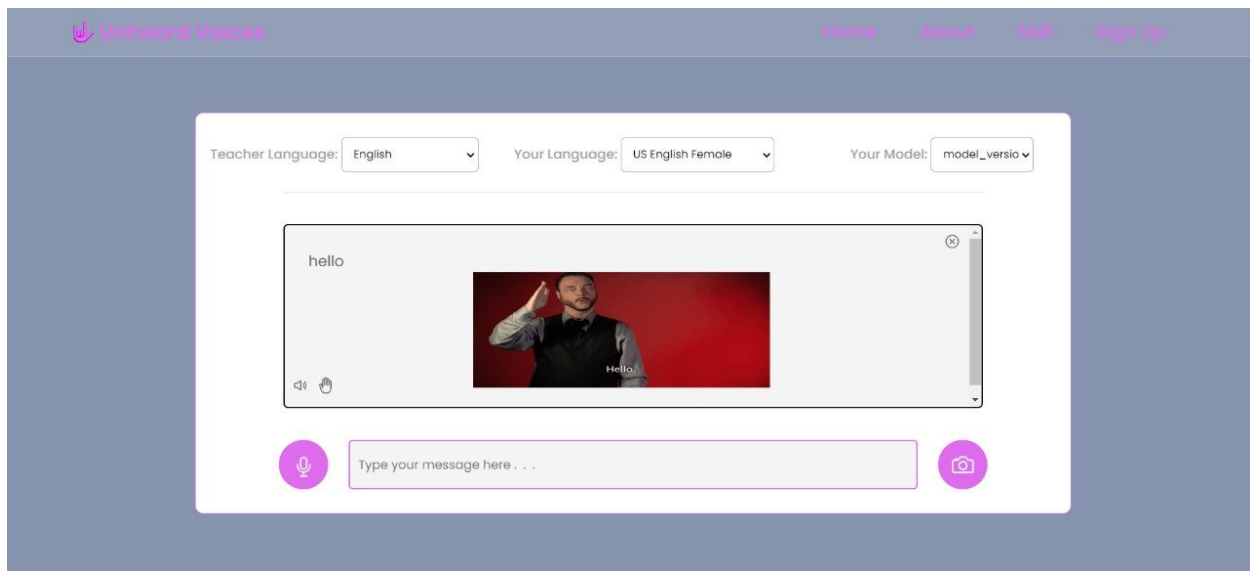


Figure 5.4 skill page

Users have the flexibility to record their voice and convert it into text messages, facilitating seamless communication between individuals with different communication preferences. This feature promotes accessibility and inclusivity by accommodating diverse user needs and fostering meaningful interactions.

The About page grants users access to their device's camera, empowering them to engage in live video conversations and practice sign language gestures. Camera integration enhances the learning experience and encourages interactive communication, allowing users to express themselves visually and creatively.

An innovative feature of the About page is the ability to convert voice recordings into GIF sequences. Users can capture their sign language expressions and convert them into animated GIFs, facilitating visual communication and expression in a dynamic and engaging manner.

Building on the foundation of inclusivity, the About page also offers sign language-to-voice translation functionality. Users with hearing impairments can communicate through sign language via the camera,

and the system translates their gestures into voice output, enabling seamless communication with normal individuals.

5.2 Mobile App Design :-

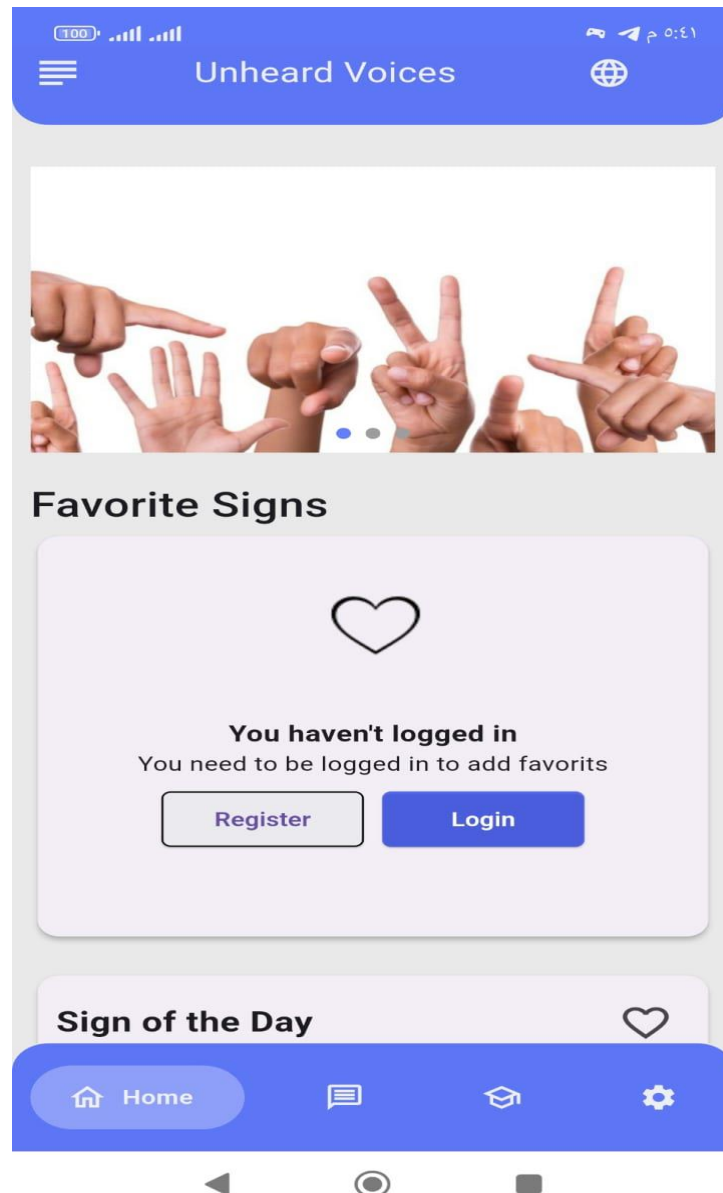


Figure 5.5 Home page for mobile application

- The Home Page prominently features options for users to log in to their existing accounts or register for new ones. This streamlined process allows users to access personalized features and track their progress in sign language learning
- An integral component of the Home Page is the advertisement section, which showcases promotional content highlighting the app's mission to empower both normal individuals and those with hearing impairments to learn sign language. These advertisements serve as a visual representation of the app's commitment to inclusivity and accessibility.
- Users can easily access the login and registration functionalities directly from the Home Page. By logging in or registering for an account, users gain access to personalized features and content tailored to their learning needs.
- The Home Page in the Unheard Voices mobile application prominently features advertisements promoting inclusivity and accessibility in sign language education. These advertisements showcase the app's commitment to empowering both the deaf and hard of hearing community and normal individuals to learn sign language. Additionally, the app highlights the significance of sign language GIFs as a means of communication and expression for all users

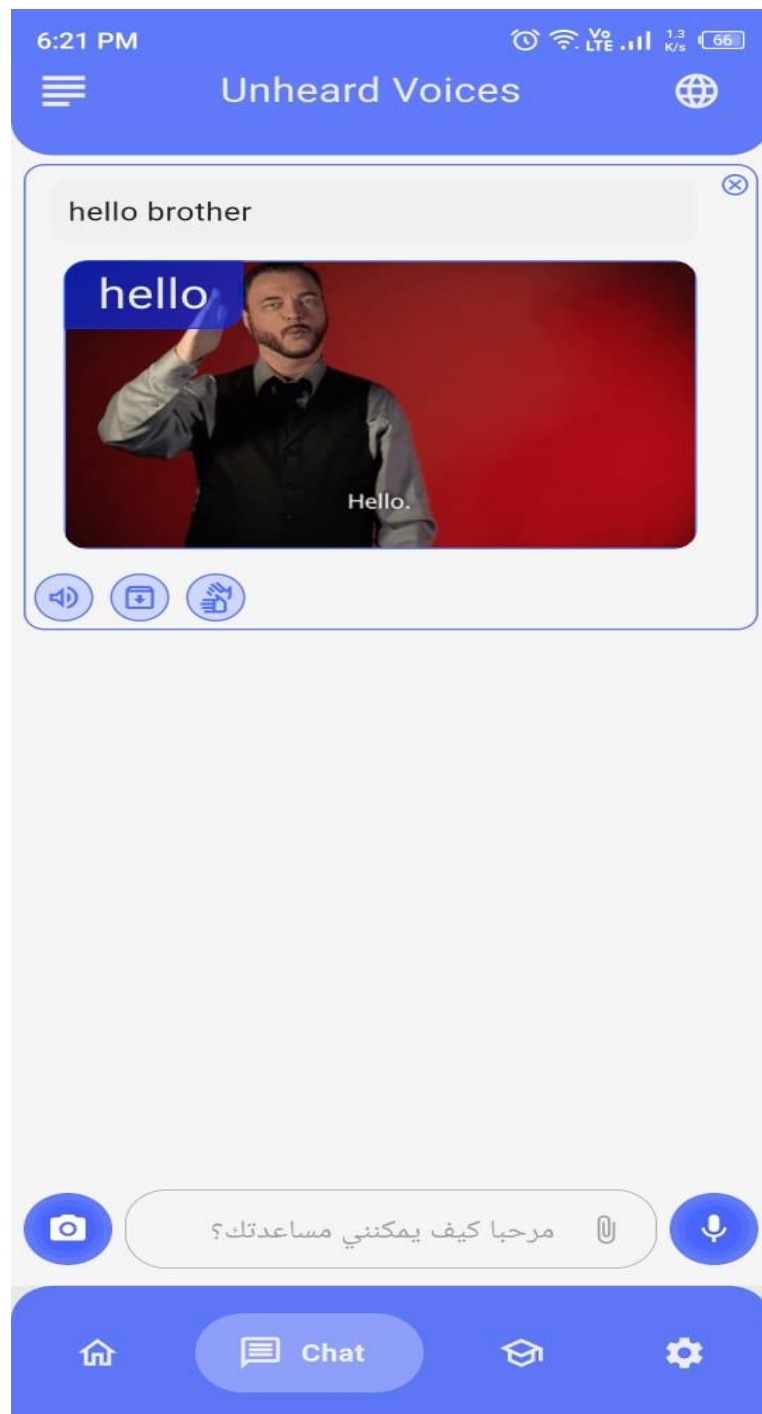


Figure 5.6 chat page

Voice Recording and Text Messaging:

Users have the option to record their voice and convert it into text messages, facilitating seamless communication between individuals with different communication preferences. This feature promotes accessibility and inclusivity by accommodating diverse user needs and fostering meaningful interactions.

Camera Access:

The About Page grants users access to their device's camera, empowering them to engage in live video conversations and practice sign language gestures. Camera integration enhances the learning experience and encourages interactive communication, allowing users to express themselves visually and creatively.

Voice-to-GIF Conversion:

An innovative feature of the About Page is the ability to convert voice recordings into GIF sequences. Users can capture their sign language expressions and convert them into animated GIFs, facilitating visual communication and expression in a dynamic and engaging manner.

Sign Language-to-Voice Translation:

Building on the foundation of inclusivity, the About Page also offers sign language-to-voice translation functionality. Users with hearing impairments can communicate through sign language via the camera,

and the system translates their gestures into voice output, enabling seamless communication with normal individuals.

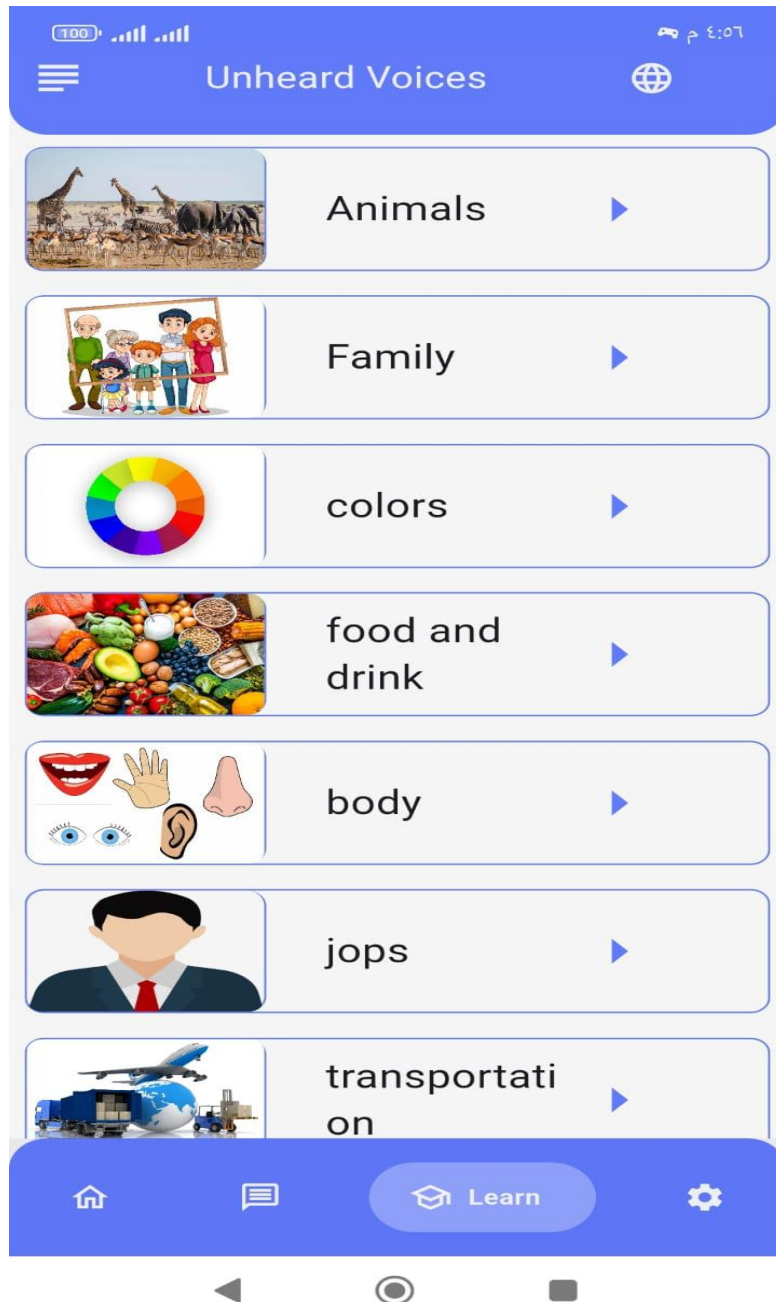


Figure 5.7 learning page (categories)

Structured Learning Categories:

The Categories feature organizes sign language learning content into structured categories, covering a wide range of topics and vocabulary. Users can navigate through different categories such as alphabet, numbers, greetings, emotions, and more, making it easy to access relevant learning materials based on their interests and learning goals.

One of the standout features of the Categories and Learning Model is its integration with the device's camera. Users can utilize the camera to practice sign language gestures and receive real-time feedback on their performance. The app analyzes the user's gestures and provides feedback on correctness, helping users improve their sign language skills effectively.

- **ASL Model detection (Ai) :-**

1. Import and Install Dependencies :-

In a typical machine learning project, these imports are essential for various tasks:

File operations: Using `os` for handling file paths and directory creation.

Image and video processing: Using `cv2` for reading, displaying, and manipulating images and videos.

Numerical computations: Using numpy for handling arrays and performing mathematical operations.

Building neural networks: Using Keras to create, compile, and train models, particularly for sequential data with LSTM.

Data preparation and evaluation: Using scikit-learn for splitting the dataset and evaluating the model's performance with appropriate metrics.

2. Setup Folders for Collection :-

Data Collection:

You gather video clips for each action (e.g., people interacting as friends, a person holding a gun, healthcare-related activities). Each video clip is split into sequences of a fixed length (15 frames in this case).

Data Organization:

You organize the collected data into folders corresponding to each action.

Each folder contains 500 sequences, with each sequence being 15 frames long.

Model Training:

The organized data is used to train a machine learning model. The model learns to recognize patterns associated with each action based on the provided sequences.

3. **Preprocess Data and Create Labels and Features :-**

This setup is typical in machine learning projects where you need to preprocess and organize data before feeding it into a model. Here's a practical breakdown of what this code is preparing:

Label Mapping:

Each action is assigned a unique integer, facilitating easy reference and processing in the machine learning pipeline.

Data Loading and Reshaping:

The code loads data from .npy files. These files are likely preprocessed and stored sequences of frames (e.g., video data).

Reshaping the data is crucial for standardizing the input format. Converting (30, 543, 3) to (30, 1629) makes it easier to handle in neural network models.

Organizing Data:

sequences (X data) and labels (y data) are collected and organized into arrays that can be directly used for training.

Converting labels to one-hot encoding is essential for classification tasks, enabling the model to output probabilities for each class.

4. represents the structure and dimensions of the data stored in the NumPy array :-

In the context of machine learning, especially for tasks involving sequential data like video or time series, understanding the shape of your input data is crucial for designing and training your models effectively.

Understanding these dimensions helps in structuring the data correctly for machine learning models, ensuring that each sequence of frames is appropriately represented and can be effectively learned by the model

5. splitting data into training and testing sets :-

Model Training:

The training set (X_train and y_train) is used to train the machine learning model. The model learns from this data by adjusting its parameters to minimize the prediction error.

Model Evaluation:

The testing set (`X_test` and `y_test`) is used to evaluate the model's performance. This helps to understand how well the model generalizes to new, unseen data.

Validation of the Process:

Splitting data into training and testing sets is crucial to prevent overfitting and to ensure that the model can generalize well to new data.

Using `train_test_split` ensures a random and representative split of the data, maintaining the distribution of classes between training and testing sets.

6. the shape of a NumPy array or a similar object :-

- `actions.shape[0]` gives the size of the first dimension of the actions array.
- If `actions` is a 1D array or list of action labels, `actions.shape[0]` returns the number of action labels.
- This information is useful for various tasks, including looping through actions, setting model parameters, and preparing data for machine learning tasks.

7. Build and Train LSTM Neural Network :-

Model Architecture: The model is built using a Sequential API with stacked LSTM layers followed by dense layers.

Three LSTM layers with 64, 128, and 64 units respectively.

Two dense layers with 64 and 32 units.

Final dense layer with a number of units equal to the number of action classes, using softmax activation.

Training: The model is compiled with the Adam optimizer and categorical crossentropy loss function. It is trained for 100 epochs on the training data.

Output: The model summary provides an overview of the network architecture, including the layers, output shapes, and the number of parameters.

This setup is designed for a sequence classification task, such as recognizing actions in video clips where each sequence consists of a series of frames, each represented by a feature vector.

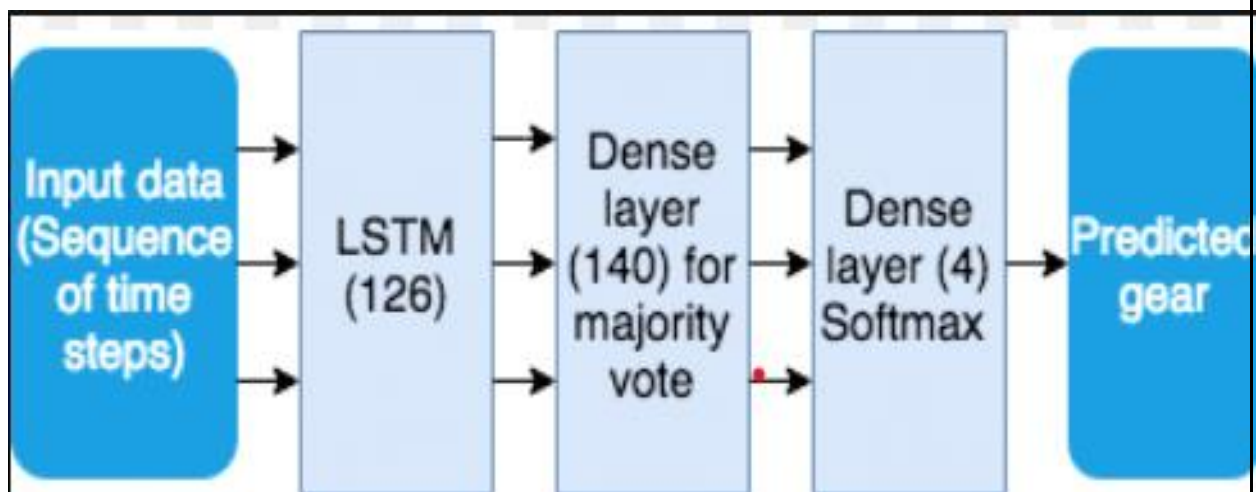


Fig. 5.8: Network architecture for the LSTM model

A Dense layer, also known as a fully connected layer, It is called "dense" because every neuron in a Dense layer is connected to every neuron in the preceding layer.

- **Feature Extraction and Aggregation:**

- The LSTM layers extract temporal features from the sequential input data, capturing patterns and dependencies between signs in sign language.
- However, the output of the LSTM layers may be high-dimensional and complex.
- The Dense layer(s) following the LSTM layers help to further process and aggregate these features, summarizing the information learned by the LSTM layers into a more compact representation.

- **Regularization:**

- Dense layers can act as a form of regularization by preventing overfitting and improving the generalization ability of the model.

- **Final Classification:**

- The Dense layer(s) culminate in the output layer, which performs the final classification of sign language gestures.

8. Evaluation using Confusion Matrix and Accuracy :-

Prediction ($\hat{y}$): Model predicts probabilities for each class for the test data.

True Labels (y_{true}): Convert one-hot encoded true labels to class labels.

Predicted Labels ($\hat{y}$): Convert predicted probabilities to class labels.

Accuracy Calculation: Compare true labels and predicted labels to compute accuracy, indicating the proportion of correctly predicted labels.

This process evaluates how well the trained model performs on the test data by providing an accuracy score, which is a common metric for classification tasks.

9. **save weights :-**

Saving the model after training is essential for several reasons:

Reusability: Saved models can be loaded later and reused for making predictions on new data without needing to retrain the model.

Sharing: Models can be easily shared with others, allowing collaborators or users to use the model without having to train it themselves.

Deployment: Saved models can be deployed in production environments for making predictions in real-time applications.

Saves the trained model's architecture and weights to the specified file path in HDF5 format.

Saving models allows for easy reuse, sharing, and deployment of trained models in various applications.

3.3. LSTM Model Setup :-

An alternative method to find time-lag value is to use an optimization algorithm such as GA.

The GA can be applied on the features space of the predictive model to search for the optimal solution. Such method has the ability to find a good subset of solutions

without investigating all possible subset of solutions. In this paper, the GA is applied on a predictive model developed by LSTM to search for optimal value of time-lag length feature.

The reason of using LSTM model is the ability to perform a multi-step estimation given a data time window which makes it suitable for time-series forecasting.

The LSTM model used in this paper for PN concentration forecasting consists of one input layer, one output layer and three hidden layers. The configuration details of the LSTM hyperparameters are illustrated below and the overall architecture is shown in

Figure 4

The Sequential Long Short-Term Memory (LSTM) architecture is a type of recurrent neural network (RNN) architecture designed specifically for handling sequential data. It's a powerful framework widely used in various fields such as natural language processing, time series analysis, speech recognition, and more. Let's dive into the key aspects of this architecture:

1. Recurrent Neural Networks (RNNs):

RNNs are a class of neural networks that process sequential data by maintaining a hidden state that evolves over time as new input is fed into the network. They are particularly suitable for tasks where the input data's temporal order is crucial.

2. Long Short-Term Memory (LSTM) Cells:

LSTM cells are specialized units within LSTM layers that address the vanishing gradient problem faced by traditional RNNs. They achieve this by introducing gates that regulate the flow of information through the network. Each LSTM cell typically includes:

Memory Cell: Maintains information over time and is capable of learning long-term dependencies in the data.

Input Gate: Controls the flow of new information into the memory cell.

Forget Gate: Determines which information to discard from the memory cell.

Output Gate: Manages the flow of information from the memory cell to the output of the layer.

1. Sequential Long Short-Term Memory (LSTM) Architecture:

In the Sequential LSTM architecture, multiple LSTM layers are stacked sequentially to form a deep neural network. Each LSTM layer processes the input sequence and passes its output to the next layer, allowing the network to learn hierarchical representations of the data.

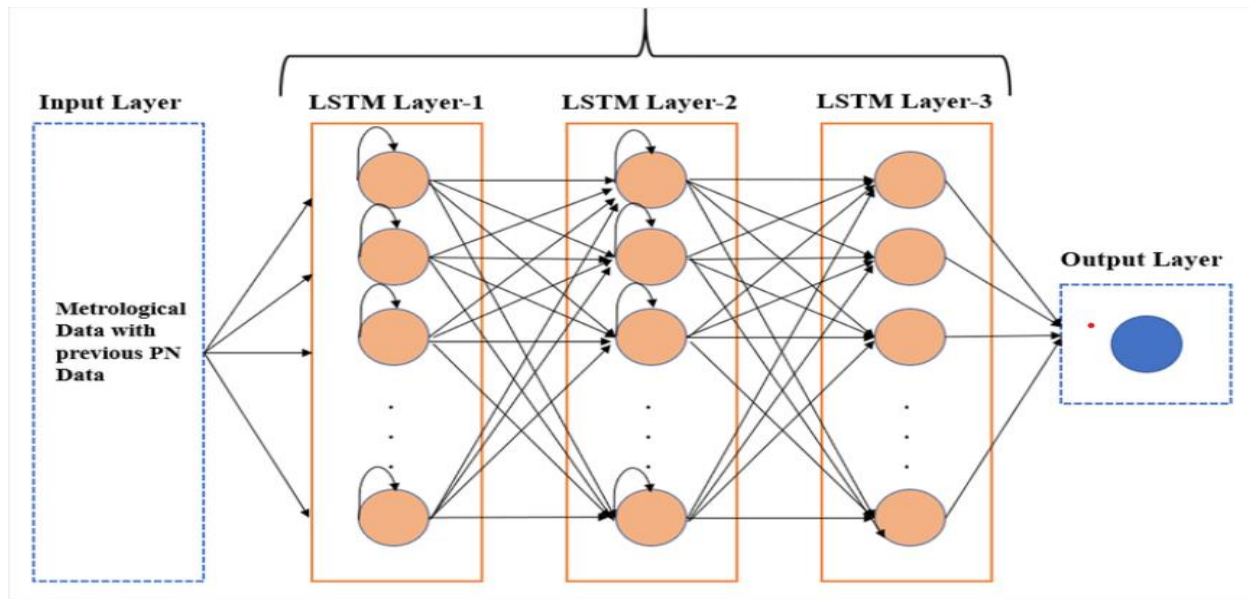


Figure 5.9. Sequential Long Short-Term Memory (LSTM) architecture

5.3 Code Snippets :-

```
function startSpeechToText() {
  try {
    recognition = new SpeechRecognition();
    recognition.lang = inputLanguage.value;
    recognition.interimResults = true;
    recordbtn.classList.add("recording");
    recognition.start();
    recognition.onresult = (event) => {
      const speechResult = event.results[0][0].transcript;
      if (event.results[0].isFinal) {
        | createNewRecoSection(speechResult);
      }
    };
    recognition.onspeechend = () => {
      startSpeechToText();
    };
    recognition.onerror = (event) => {
      stopRecording();
      if (event.error === "no-speech") {
        alert("No speech was detected. Stopping...");
      } else if (event.error === "audio-capture") {
        alert("No microphone was found. Ensure that a microphone is installed.");
      } else if (event.error === "not-allowed") {
        alert("Permission to use the microphone is blocked.");
      } else if (event.error === "aborted") {
        alert("Listening Stopped.");
      } else {
        alert("Error occurred in recognition: " + event.error);
      }
    };
  } catch (error) {
    recording = false;
    console.log(error);
  }
}
```

Fig 5.10 speech to text

This function initializes and manages speech recognition using the Web Speech API. It sets up language, interim results, and starts the recognition process while updating the UI to indicate recording. It processes speech results, handles end-of-speech by restarting the recognition, and manages errors with appropriate alerts. Any errors during initialization are caught and logged.

```

}
const camSection = document.querySelector(".cam-section")
const webCamElement = document.getElementById("webCam");
const camBtn = document.querySelector(".btn-cam");
const canvasElement = document.getElementById("canvas");
let isCameraOn = false;
const webcam = new Webcam(webCamElement, "user", canvasElement);
var numberFrames = 0;
const frameCount = document.getElementById('frameCount');
const frameCount2 = document.getElementById('frameCount2');
let previousPrediction = '';
let continueSendingFrames = true;
console.log('isCameraOn')
async function startFPS() {
  try {
    var video = document.getElementById('webCam');
    var canvas = document.getElementById('canvas');
    var ctx = canvas.getContext('2d');
    canvas.width = 480;
    canvas.height = 640;
    await video.play();
    ctx.drawImage(video, 0, 0, canvas.width, canvas.height);
    var imageData = ctx.getImageData(0, 0, canvas.width, canvas.height);
    var bytes = new Uint8Array(imageData.data);
    var serverURL = 'http://192.168.245.224:3000/upload';
    if (continueSendingFrames) {
      await sendFramesAndPredict(serverURL, bytes);
    }
  } catch (error) {
    console.error(error);
  } finally {
    var video = document.getElementById('webCam');
    if (!video.paused && !video.ended && continueSendingFrames) {
      requestAnimationFrame(startFPS);
    }
  }
}
}

```

Fig 5.11 API Call in Webcam Functionality

The API call is part of the webcam functionality where frames from the webcam are sent to a server for processing. The `startFPS` function captures frames from the webcam, converts them to bytes, and calls the `sendFramesAndPredict` function.

```

async function sendFramesAndPredict(serverURL, bytes) {
  try {
    const response = await fetch(serverURL, {
      method: 'POST',
      body: bytes,
    });

    console.log('Response Status:', response.status);

    if (!response.ok) {
      console.error('Server returned an error:', response.status);
      return;
    }
    const data = await response.json();
    const newPrediction = data.message;

    console.log('New Prediction:', newPrediction);
    if (newPrediction !== null && newPrediction.trim() !== '') {
      previousPrediction += newPrediction + ' ';
      inputField.value = previousPrediction.trim();
    }
  } catch (error) {
    console.error('Error:', error);
  }
}

```

Fig 5.12 Sending Frames to the Server

The sendFramesAndPredict function performs the following tasks:

- Sends a POST request to a server with binary data (frames).

- Logs the response status.

- Checks for any errors in the response and logs them.

- Parses the JSON response body to extract the prediction.

- Logs the new prediction and updates an input field with the accumulated predictions.


```
def mediapipe_detection(image, model):
    image = np.ascontiguousarray(image)
    image.flags.writeable = False
    results = model.process(image)
    image.flags.writeable = True
    return results
```

Fig 5.13 mediapipe_detection

The mediapipe_detection function is designed to take an image and a MediaPipe model, process the image with the model, and return the results. The function optimizes performance by making the image non-writeable during processing and ensuring the image data is stored contiguously in memory. This approach ensures efficient and effective processing of the image data by the MediaPipe model.

```
def create_input_model_data(all_Dataframes):
    ROWS_PER_FRAME = 543 # number of landmarks per frame
    data_columns = ['x', 'y', 'z']
    data = pd.concat(all_Dataframes, ignore_index=True)[data_columns]
    n_frames = len(data) // ROWS_PER_FRAME
    data_np = data.to_numpy().reshape(n_frames, ROWS_PER_FRAME, len(data_columns)).astype(np.float32)
    return data_np
```

Fig 5.14 create_input_model_data

The create_input_model_data function takes a list of pandas DataFrames containing landmark data and processes it into a format suitable for machine learning models

Chapter 6

Conclusions & Future Work

Conclusions

The "Unheard Voices" project has made significant strides in bridging the communication gap between deaf and hard-of-hearing individuals and those who are not familiar with sign language. Through the development of innovative software, we have created a platform that facilitates seamless communication and learning opportunities for all users. Key achievements of the project include:

Enhanced Communication: Our software enables real-time conversation between deaf and hearing individuals through a combination of sign language recognition, voice-to-sign translation, and text-based communication. This has greatly improved the inclusivity and interaction in various settings, from casual conversations to professional environments.

Sign Language Learning: The integration of a camera feature allows users to learn sign language interactively. Hearing individuals can open the camera, perform signs, and receive feedback from the software, which helps them practice and improve their skills. This feature is instrumental in promoting sign language literacy and fostering a more inclusive society.

Voice and Sign Recording: The ability to record voice and sign language sessions provides valuable resources for both personal use and educational purposes. Deaf individuals can record their signs and receive corresponding voice translations, while hearing individuals can record their spoken words and see the corresponding signs. This functionality not only aids in learning but also in creating accessible content for wider audiences.

Future Work

While the "Unheard Voices" project has made notable progress, there are several areas for future enhancement to further improve the experience and accessibility for all users:

Advanced Sign Language Recognition: Developing more sophisticated algorithms for sign language recognition to increase accuracy and expand the vocabulary of recognized signs. This includes incorporating more regional and dialectical variations of sign language.

Multilingual Support: Extending the software to support multiple sign languages and spoken languages. This will make the platform useful for a global audience and cater to the diverse linguistic needs of the deaf community.

References :-

- [1] [https://en.wikipedia.org/wiki/Flutter_\(software\)](https://en.wikipedia.org/wiki/Flutter_(software))
- [2] [https://en.wikipedia.org/wiki/Python_\(programming_language\)](https://en.wikipedia.org/wiki/Python_(programming_language))
- [3] <https://www.kaggle.com/>
- [4] <https://code.visualstudio.com/>
- [5] [https://www.researchgate.net/publication/355337314 Time-Lag Selection for Time-Series Forecasting Using Neural Network and Heuristic Algorithm](https://www.researchgate.net/publication/355337314_Time-Lag_Selection_for_Time-Series_Forecasting_Using_Neural_Network_and_Heuristic_Algorithm)
- [6] https://en.wikipedia.org/wiki/Android_Studio
- [7] <https://en.wikipedia.org/wiki/JavaScript>
- [8] https://en.wikipedia.org/wiki/Long_short-term_memory

- [9] <https://ai.google.dev/edge/mediapipe/solutions/guide>
[10] <https://www.researchgate.net/publication/332604073> Identifying cross country skiing techniques using power meters in ski poles

Citations:

- [11] <https://www.nrcpd.org.uk/supportive-roles>
[12] <https://www.pensioncorporation.com/purposeful-investments/our-esg-approach>
[13] <https://abilitynet.org.uk/news-blogs/9-useful-apps-people-who-are-ddeaf-or-have-hearing-loss>
[14] <https://www.ava.me>

Citations:

- [15] <https://surface.syr.edu/cgi/viewcontent.cgi?article=1365&context=thesis>
[16] <https://abilitynet.org.uk/news-blogs/9-useful-apps-people-who-are-ddeaf-or-have-hearing-loss>
[17] <https://www.canonsociaalwerk.eu/1790-SpO-Doveninstituut/Tellings-200-year-war-deaf-educationl.pdf>
[18] <https://www.britannica.com/science/deaf-history>
[19] <https://www.prodigium-pictures.com/ava-connecting-deaf-hearing>
[20] <https://www.indiegogo.com/projects/ava-group-conversations-made-accessible#/>
[21] <https://www.nesta.org.uk/feature/new-radicals-2018/signly/>
[22] <https://signly.co/>
[23] <https://www.cbinsights.com/company/signschool>
[24] <https://www.signschool.com/nextSteps/>